# TUTORIAL 5: THE E-COMMERCE PRODUCT CATALOG

*Object-Oriented Programming in Java — Execution Visualization & Inferences Report*

| | |
|---|---|
| **Student Name:** | Ranjeeta |
| **USN / Roll No:** | 26UG1BYAI410-T |
| **Academic Activity:** | Tutorial 5 — E-Commerce Product Catalog |
| **Core Concepts:** | Encapsulation, Inheritance, super(), Method Overloading |

## 1. Program Overview & How the Code Works

The program models an object-oriented backend catalog for an e-commerce platform. A foundational class, Product, encapsulates shared merchandise attributes (product ID, name, base price, and inventory stock). Two specialized subclasses—Electronics and Clothing—extend Product to introduce domain-specific properties: Electronics adds warrantyCost and voltage, while Clothing introduces size and fabric material.

During execution in the main() method, instances of Electronics ('Laptop') and Clothing ('Shirt') are constructed. The subclasses utilize constructor chaining via super(id, name, price, stock) to guarantee that core parent fields are safely initialized before subclass-specific variables are assigned. Subclasses then display product details by invoking super.printBaseInfo() to print foundational data without overriding. Finally, promotional discounts are calculated using method overloading, testing both percentage-based price reductions and flat dollar seasonal markdowns.

## 2. Key Inferences & Core Object-Oriented Principles

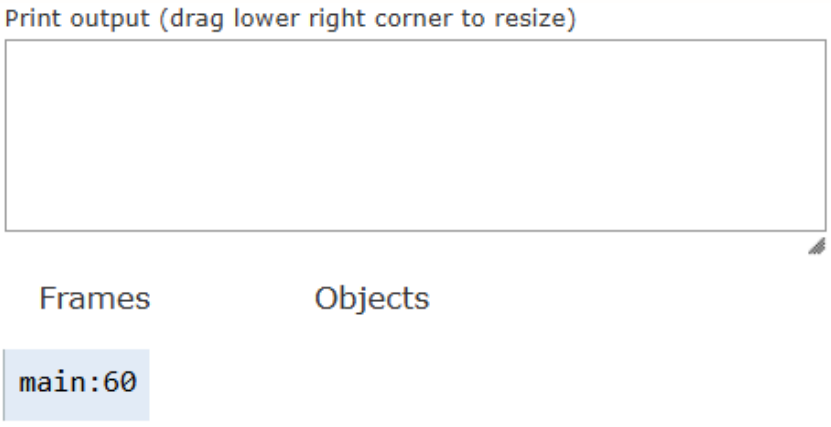
- Encapsulation & Data Security: All state variables (id, name, price, stock, warrantyCost, voltage, size, fabric) are declared private. This isolates sensitive data from unauthorized external modification, ensuring changes only occur through validated constructor routines and dedicated methods.
- Inheritance & Behavioral Reusability: The subclasses Electronics and Clothing establish an 'is-a' relationship with Product via the extends keyword. This eliminates duplicate definitions of common fields, centralizing core business logic in a single reusable base class.

- Constructor Chaining via super(): When a subclass object is instantiated, the first statement executed in its constructor is super(...). This guarantees that the superclass constructor initializes inherited private fields in the heap before the subclass constructor executes its own specific assignments.
- Parent Method Invocation via super: Inside printDetails(), calling super.printBaseInfo() triggers the parent class implementation directly. This demonstrates code reuse without method overriding, allowing subclasses to augment base behavior rather than replace it.
- Compile-Time Method Overloading: The Product class implements two applyDiscount() signatures: applyDiscount(double percent) and applyDiscount(double amount, boolean seasonal). The Java compiler selects the appropriate version at compile-time based on argument types and count, enabling flexible promotional pricing strategies.

## 3. Step-by-Step Execution Visualization & Stack/Heap Trace

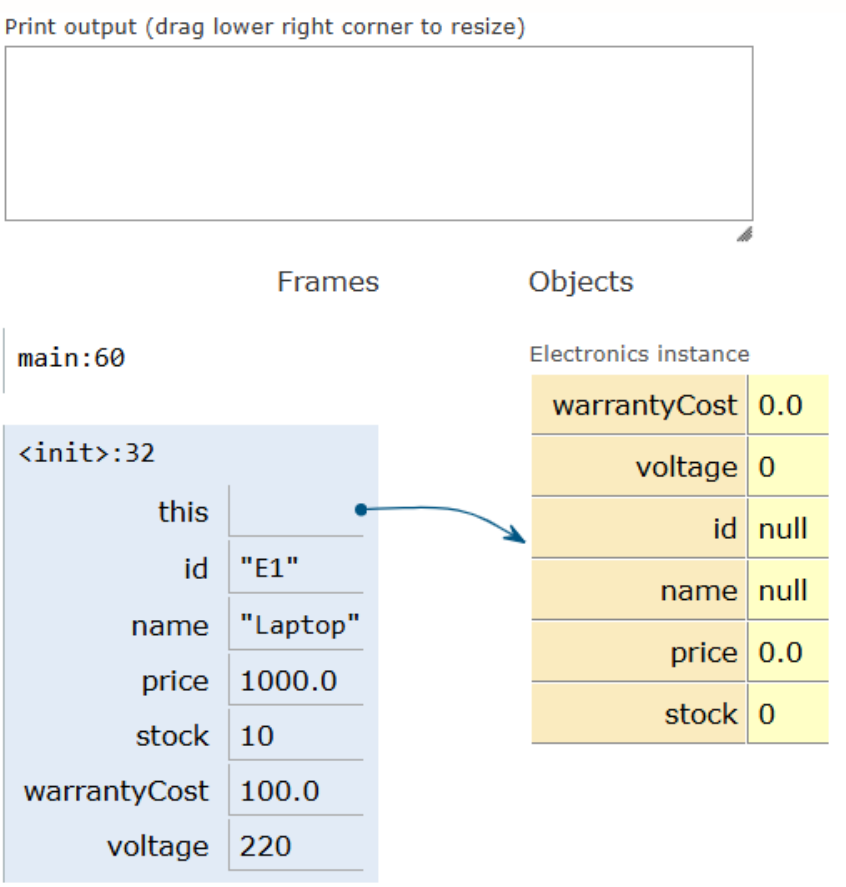
*The following section presents the visual memory state (call stack frames and heap-allocated objects) captured at each step of program execution. Each visualization is accompanied by a concise, two-line technical explanation.*

**Step 1 of 42: Main Method Entry & Execution Start**



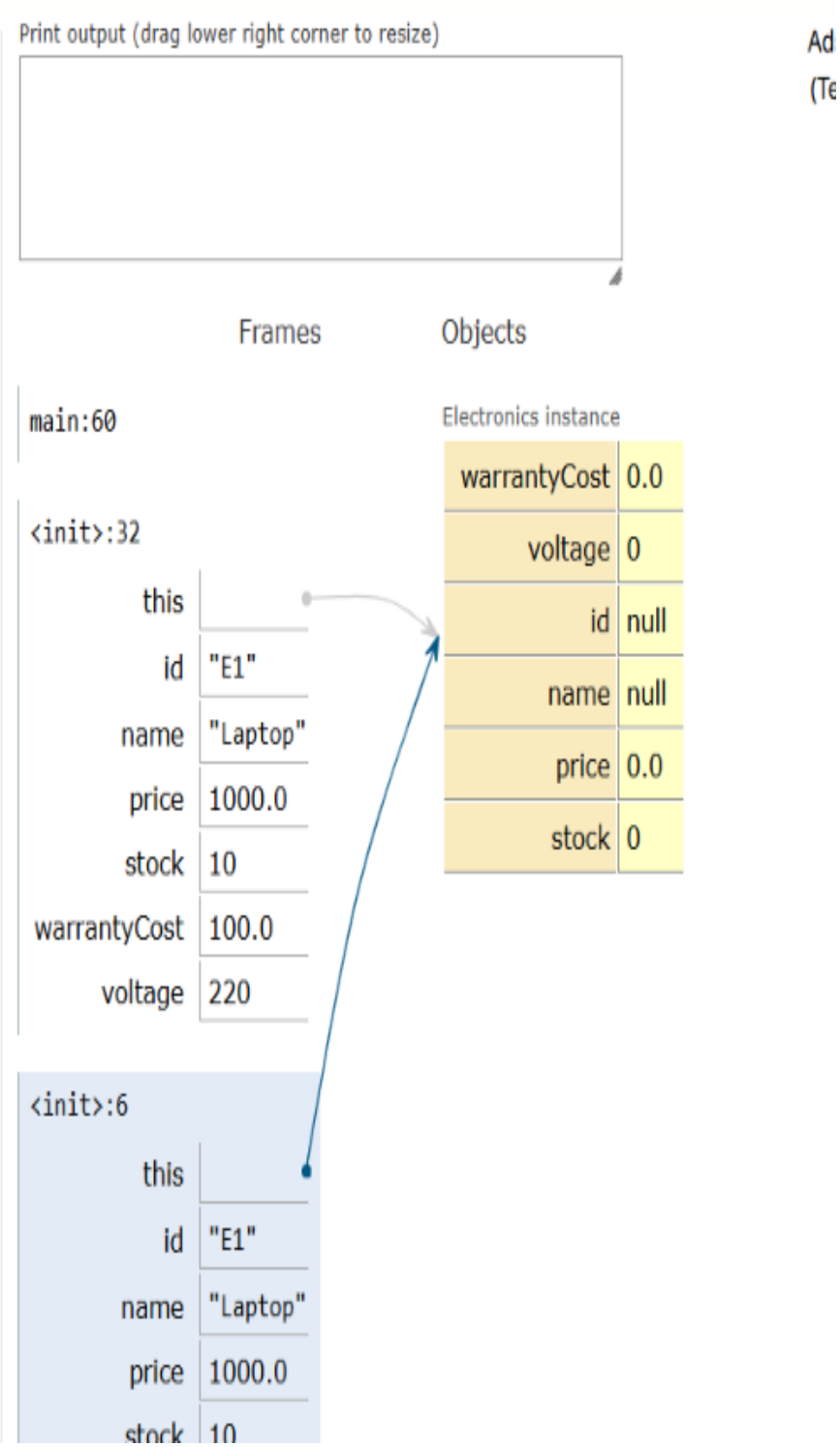

1. The JVM initializes execution by pushing the main() method frame onto the call stack.
2. Execution pauses at line 60 to begin instantiation of the Electronics laptop object.

**Step 2 of 42: Invoking Electronics Constructor & Heap Allocation**



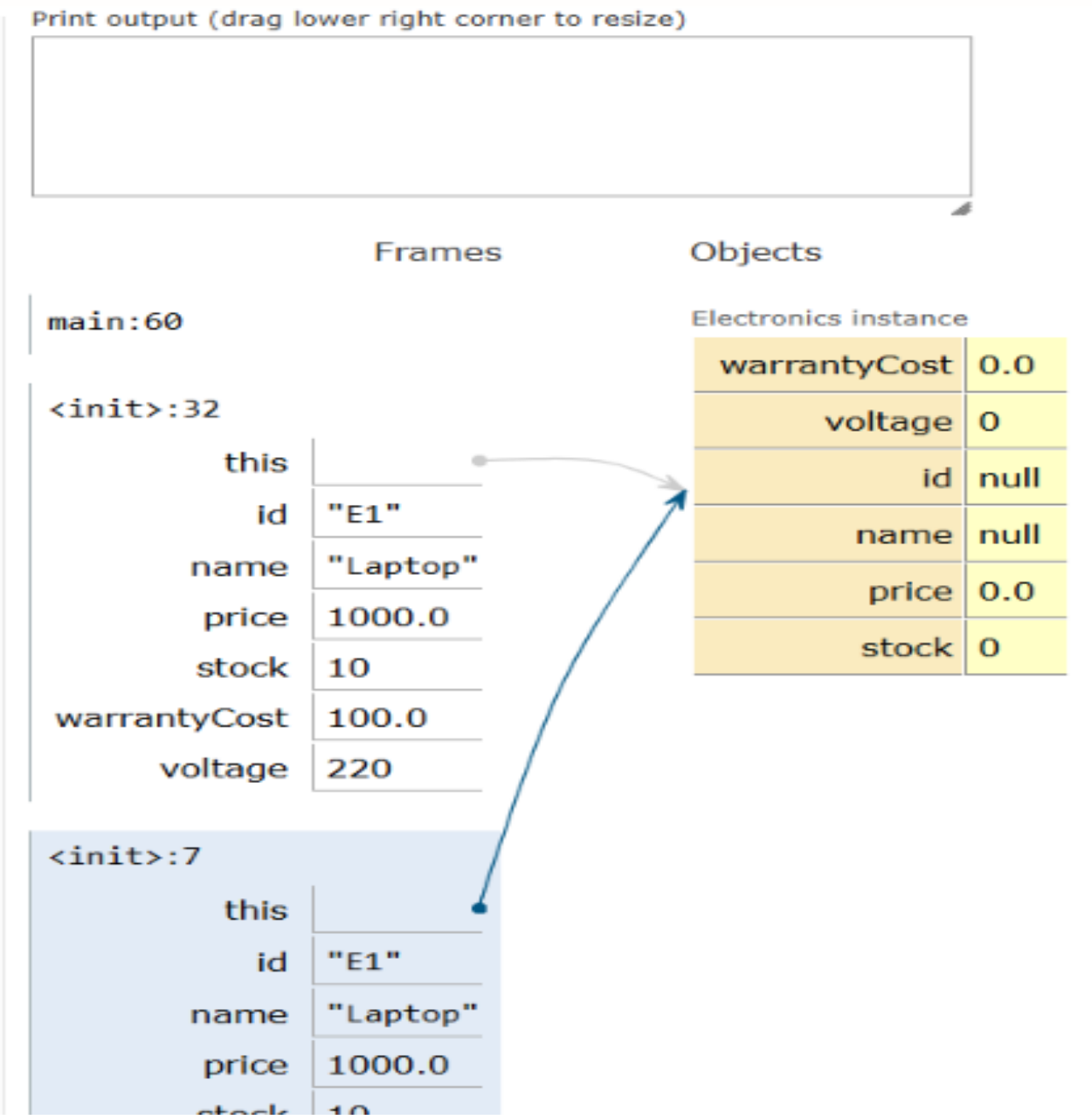

The Electronics constructor <init>:32 is called and allocated on the call stack.
An Electronics instance is created on the heap with default uninitialized fields.

**Constructor Chaining to Superclass Product**



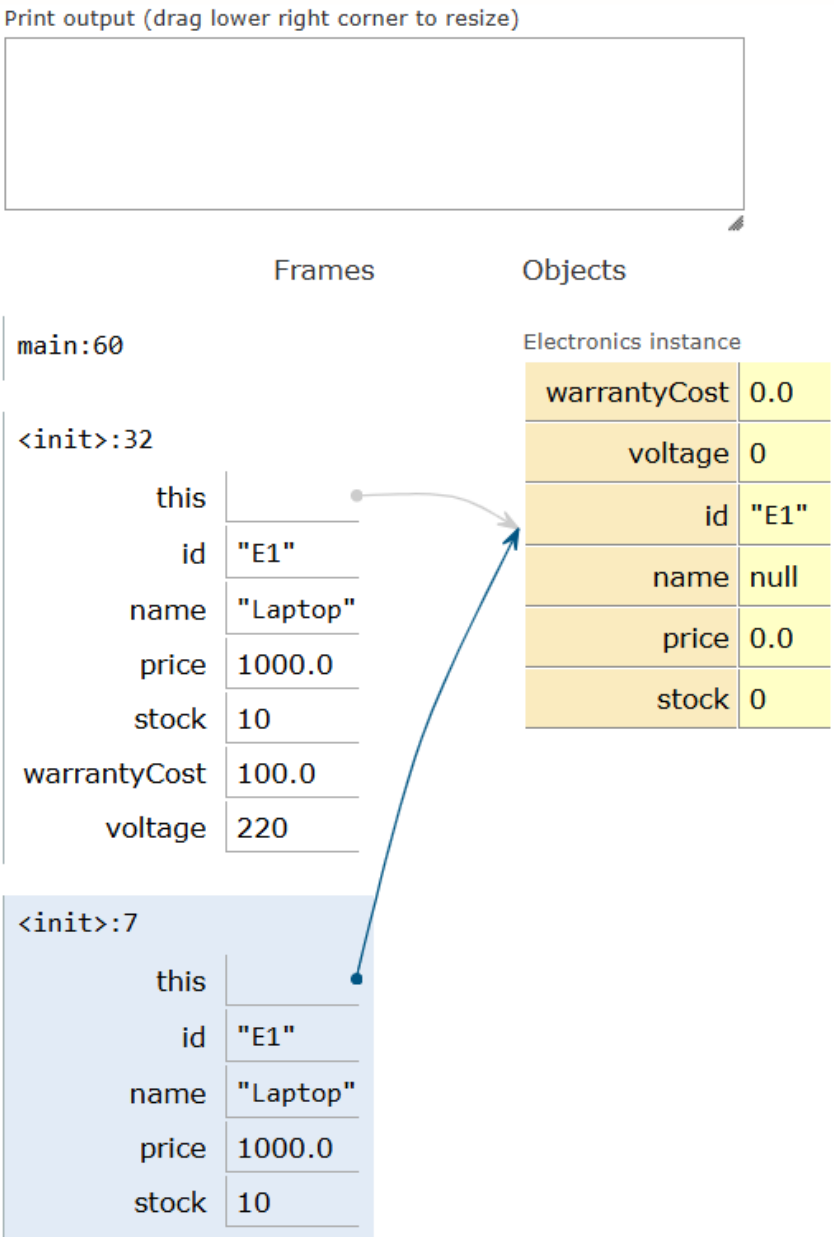

1. The constructor executes super(id, name, price, stock) to delegate initialization to Product.
2. The Product constructor frame <init>:6 is pushed onto the stack with arguments for the laptop.

**Initializing Product ID on Heap**



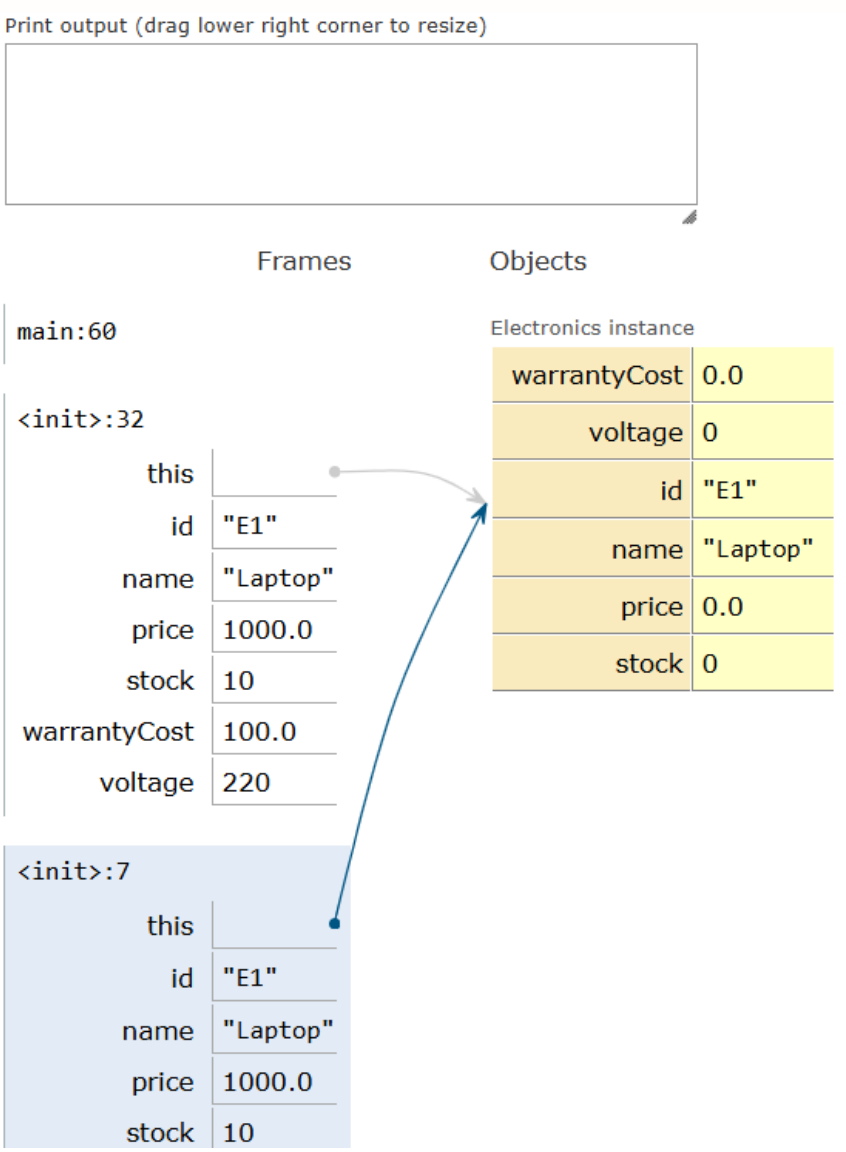

1. The parent constructor executes this.id = id to store the product identifier.
2. The id field of the Electronics heap instance is updated to reference 'E1'.

### Step 5 of 42: Initializing Product Name on Heap



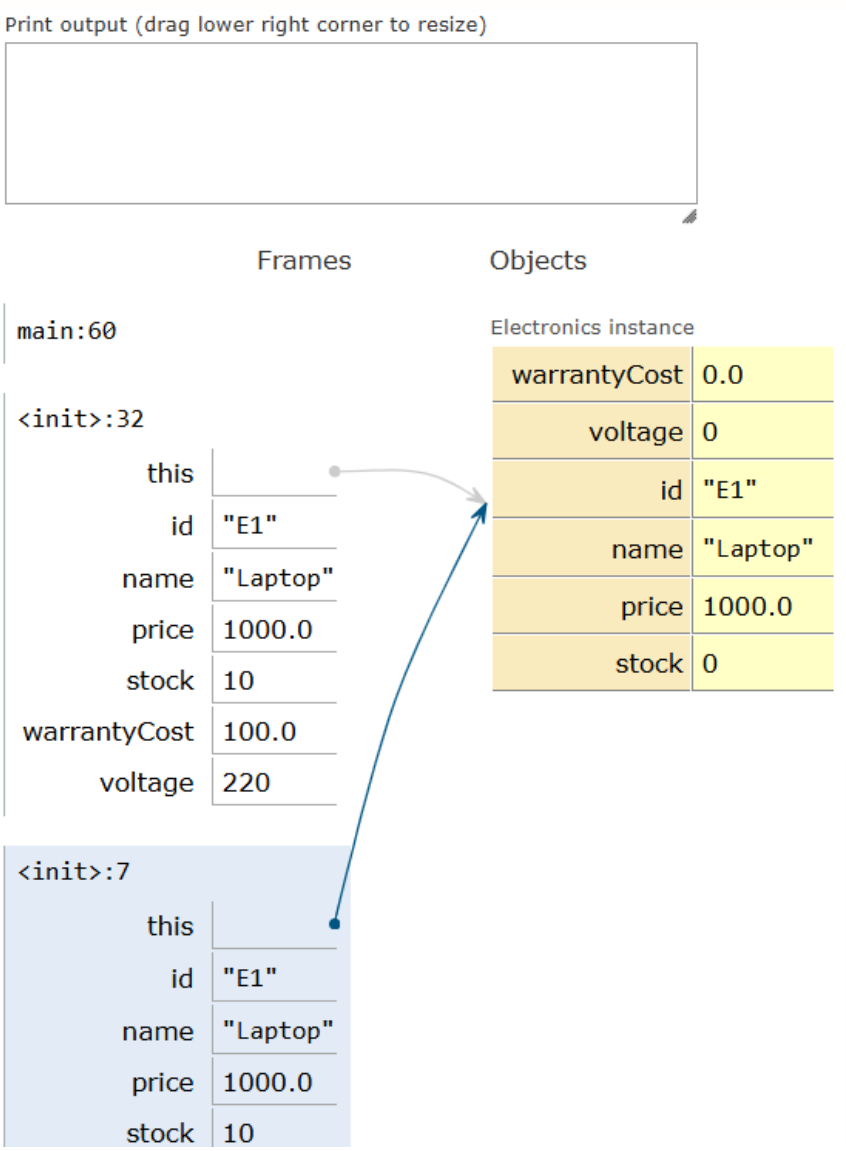

1. The parent constructor executes this.name = name to assign the product name.
2. The name field of the heap instance is updated to reference string 'Laptop'.

### Step 6 of 42: Initializing Base Price on Heap



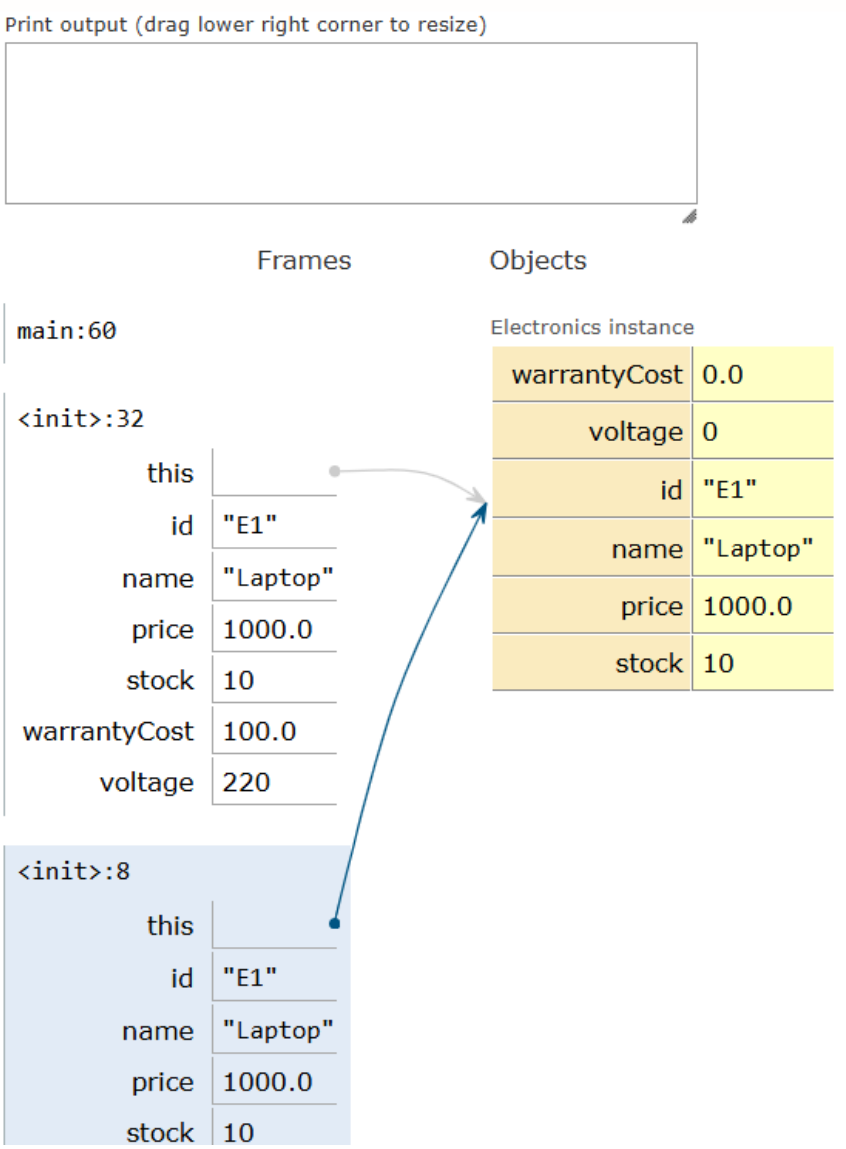

1. The parent constructor executes this.price = price to set product pricing.
2. The price field of the heap instance is assigned the floating-point value 1000.0.

### Step 7 of 42: Initializing Stock Quantity on Heap



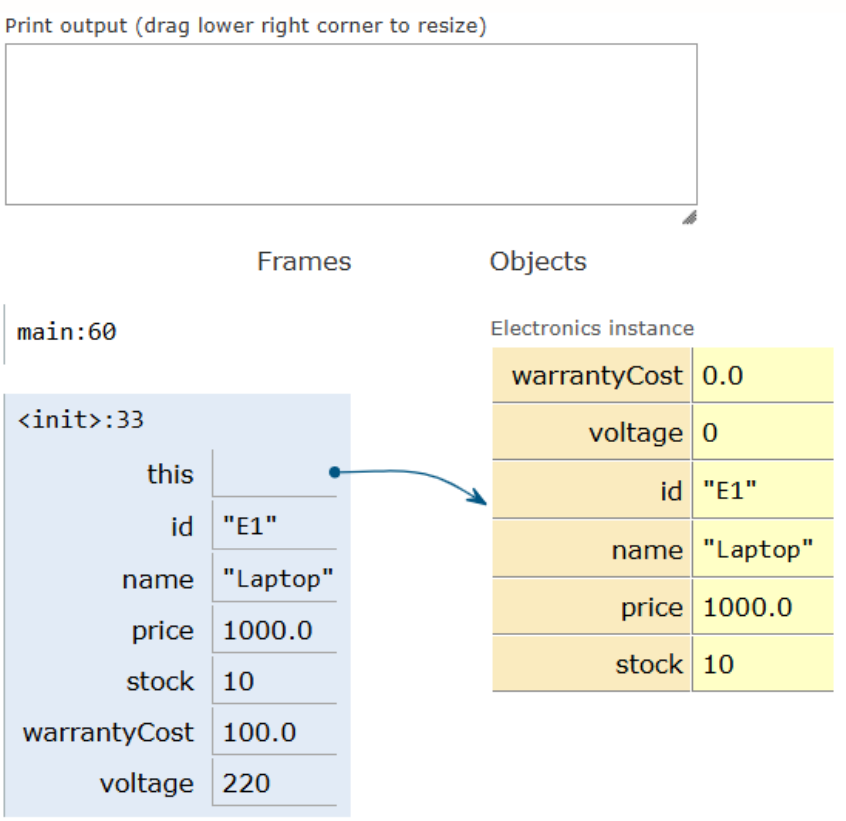

1. The parent constructor executes this.stock = stock to set inventory quantity.
2. The stock field of the heap instance is populated with integer value 10.

### Step 8 of 42: Parent Constructor Completion & Return



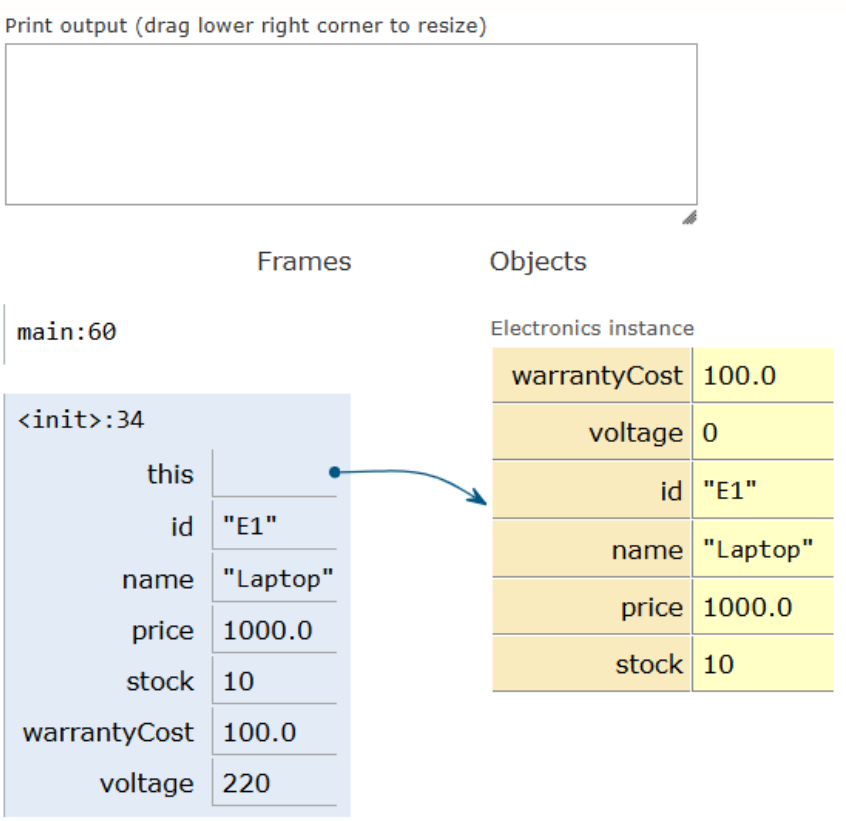

1. The Product constructor finishes its execution and pops from the call stack.
2. Control returns to the Electronics constructor <init>:33 to set subclass-specific properties.

### Step 9 of 42: Initializing Electronics Warranty Cost



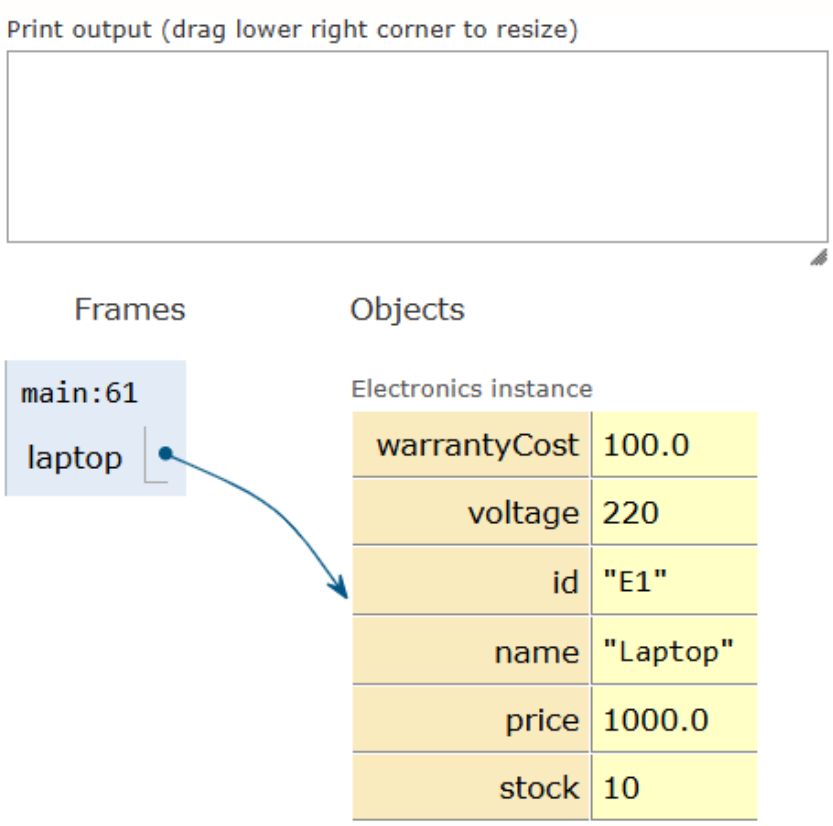

1. The Electronics constructor executes this.warrantyCost = warrantyCost.
2. The field warrantyCost of the heap instance is updated with 100.0.

### Step 10 of 42: Initializing Operating Voltage



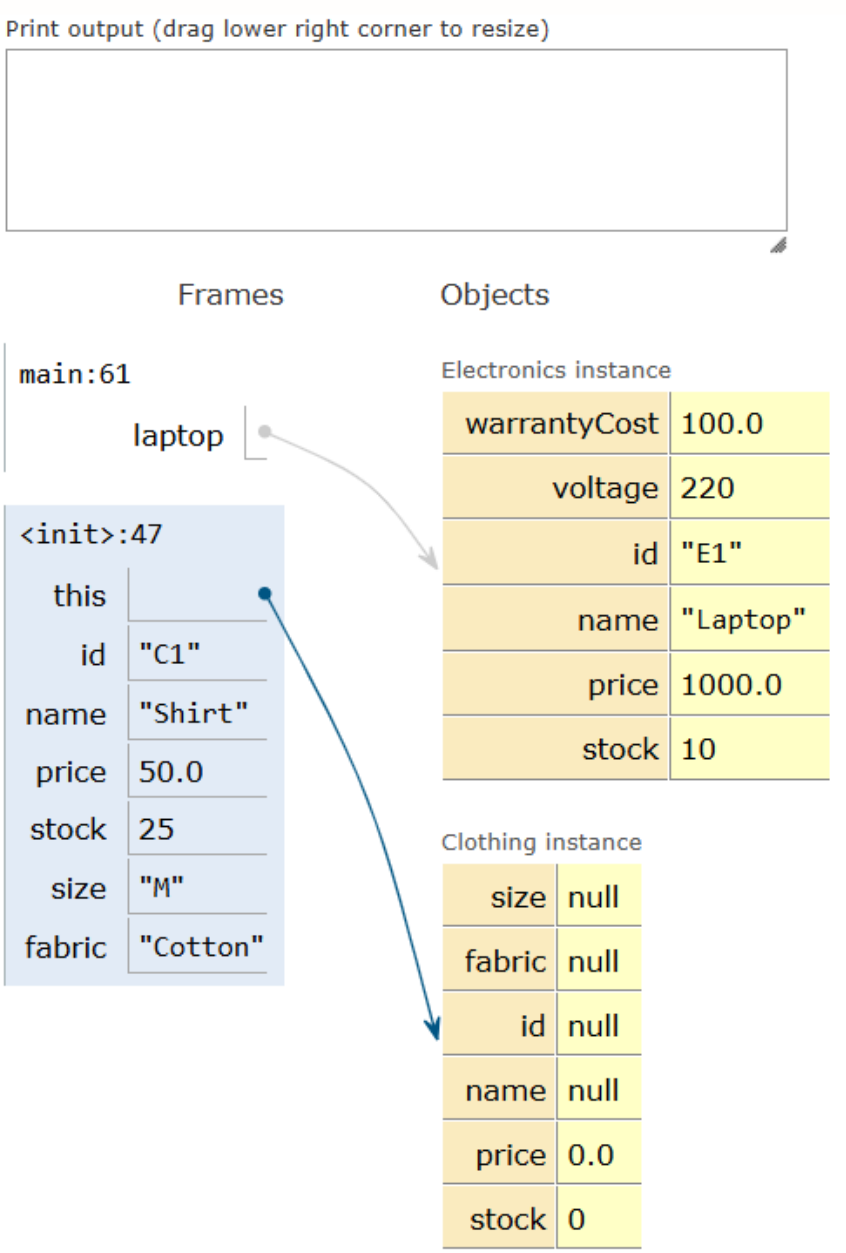

1. The Electronics constructor executes this.voltage = voltage to set electrical specs.
2. The voltage field of the heap instance is set to 220, completing laptop initialization.

### Step 12 of 42: Reference Assignment & Clothing Instantiation Start



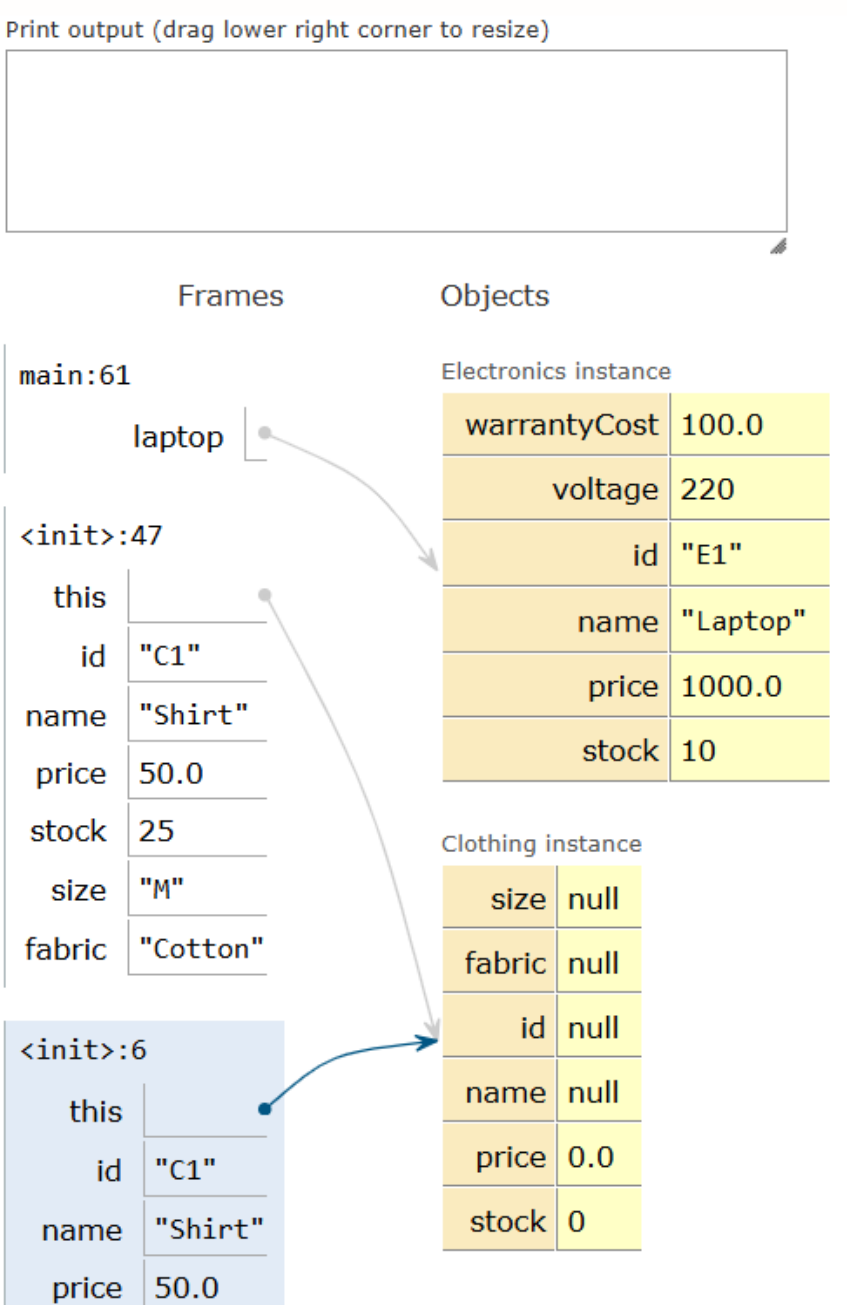

1. The laptop object reference is assigned to variable laptop in the main stack frame.
2. Execution advances to line 61 to begin instantiation of the Clothing shirt object.

### Step 13 of 42: Invoking Clothing Constructor & Heap Allocation



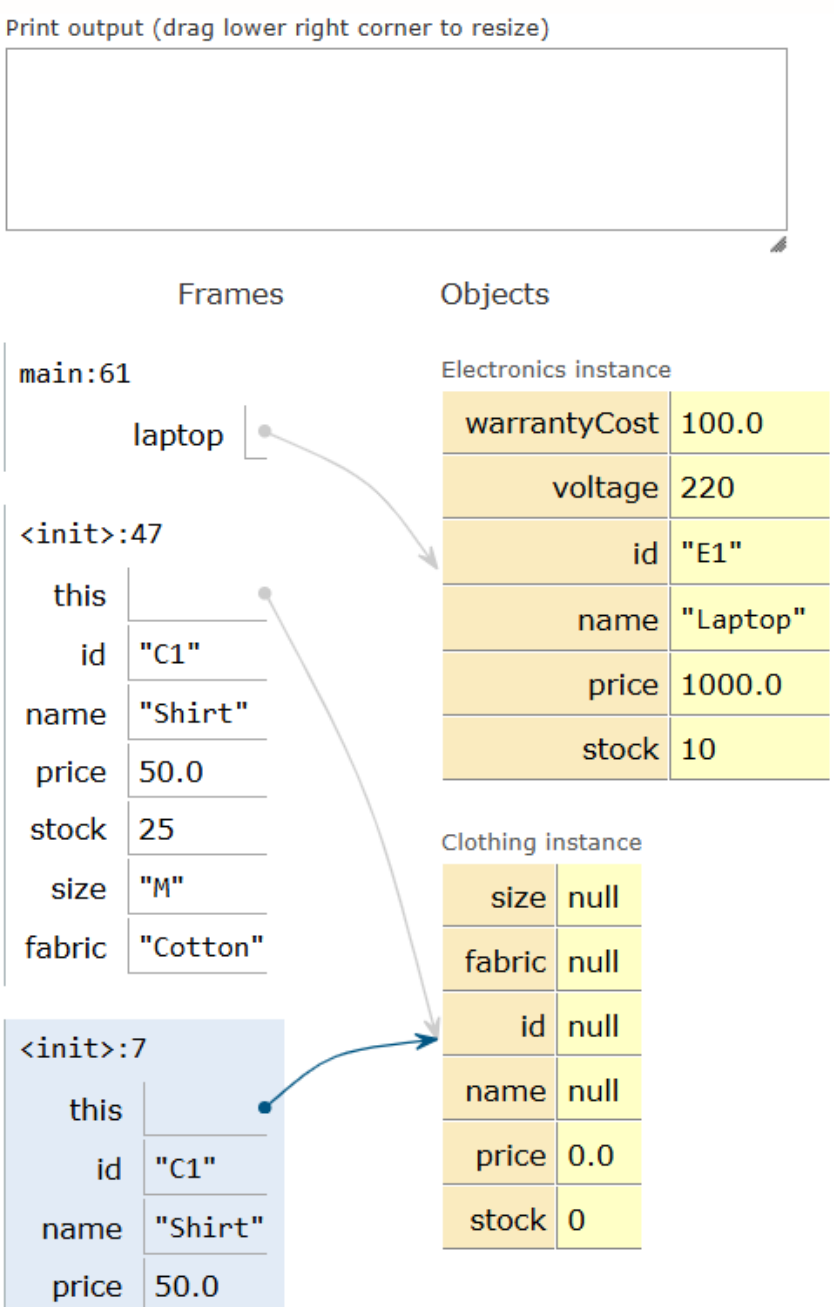

1. The Clothing constructor <init>:46 is pushed onto the stack.
2. A new Clothing instance is created on the heap with default zero and null values.

### Step 14 of 42: Constructor Chaining to Product for Clothing



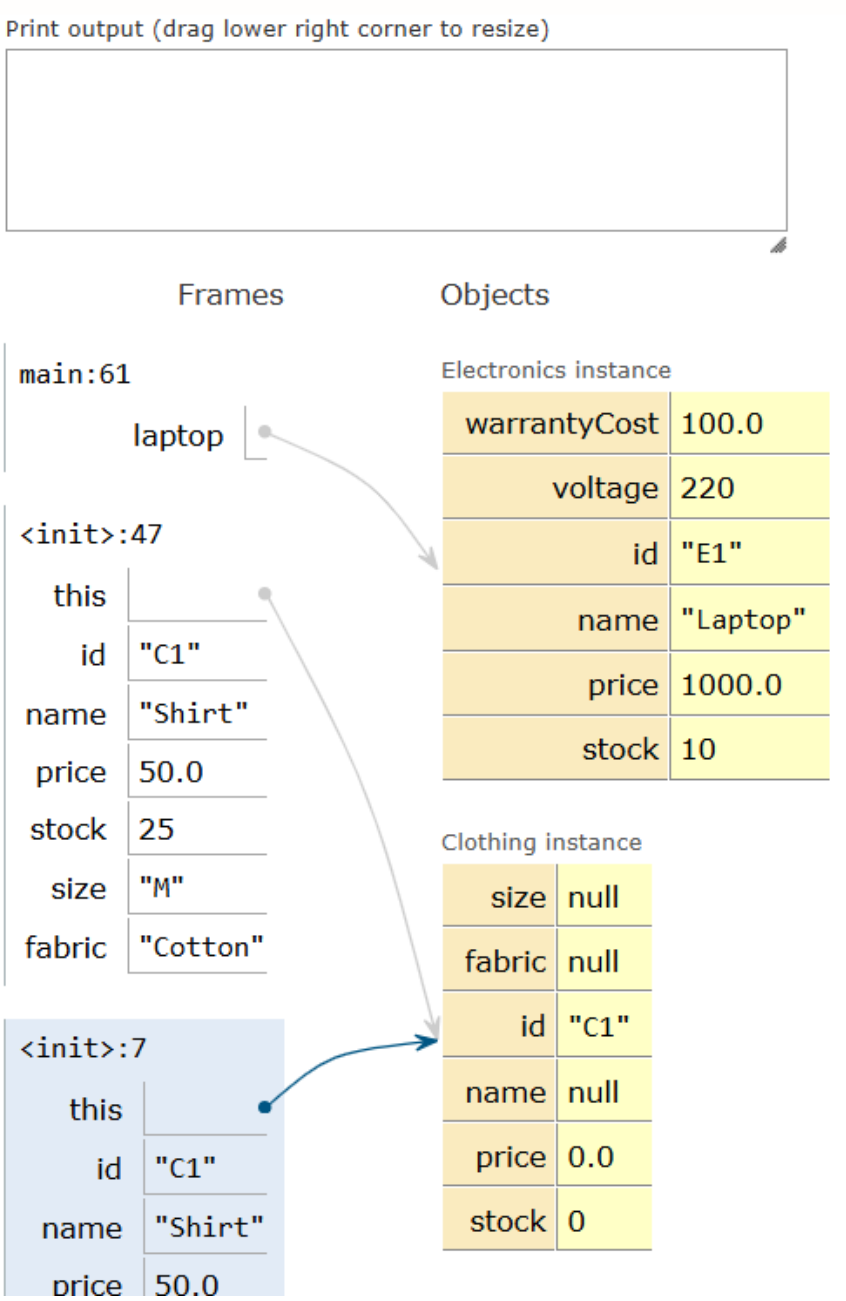

1. The statement super(id, name, price, stock) chains execution up to the Product constructor.
2. The Product constructor frame <init>:6 is pushed onto the stack with arguments for the shirt.

### Step 15 of 42: Initializing Clothing Product ID



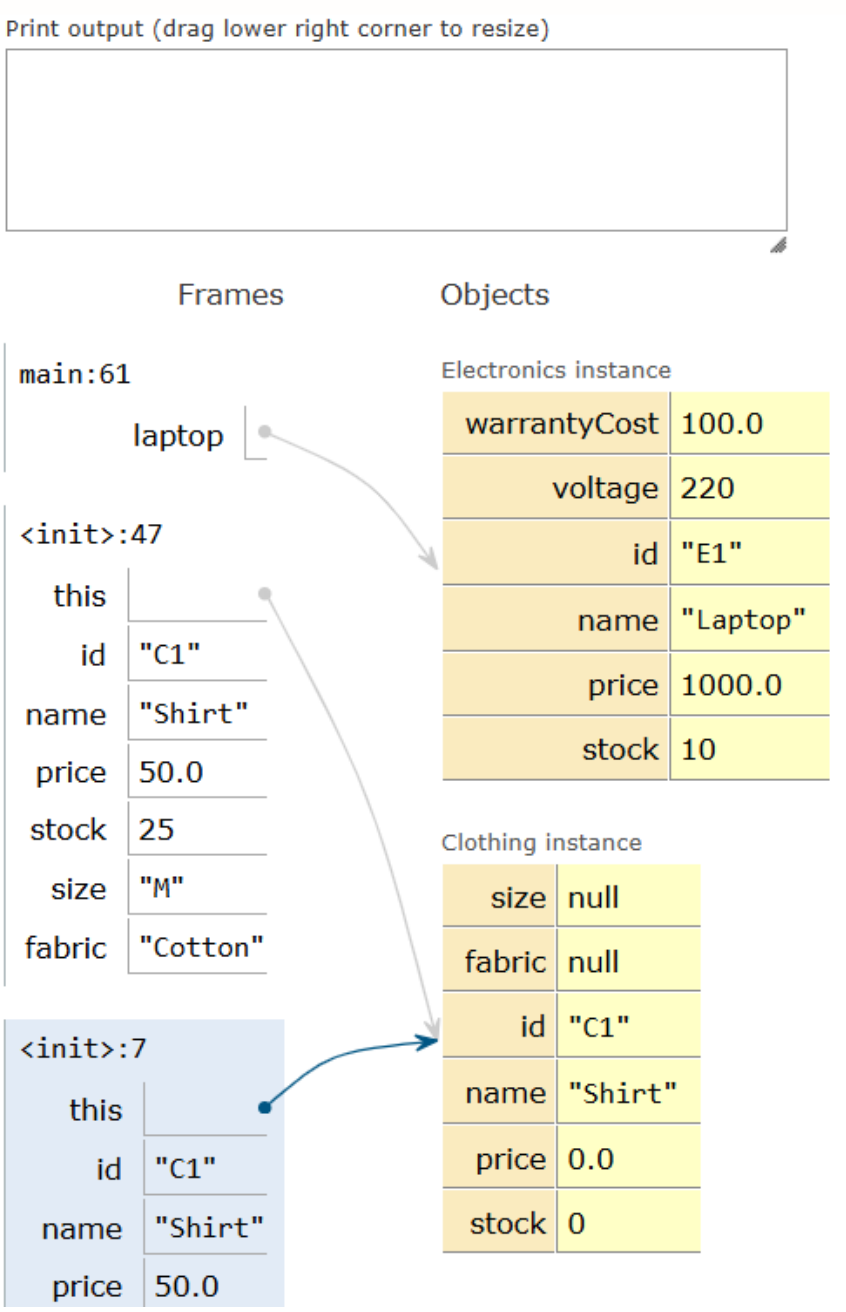

1. The parent constructor assigns the id field of the Clothing heap instance.
2. The id attribute is assigned a reference to the string 'C1'.

### Step 16 of 42: Initializing Clothing Product Name



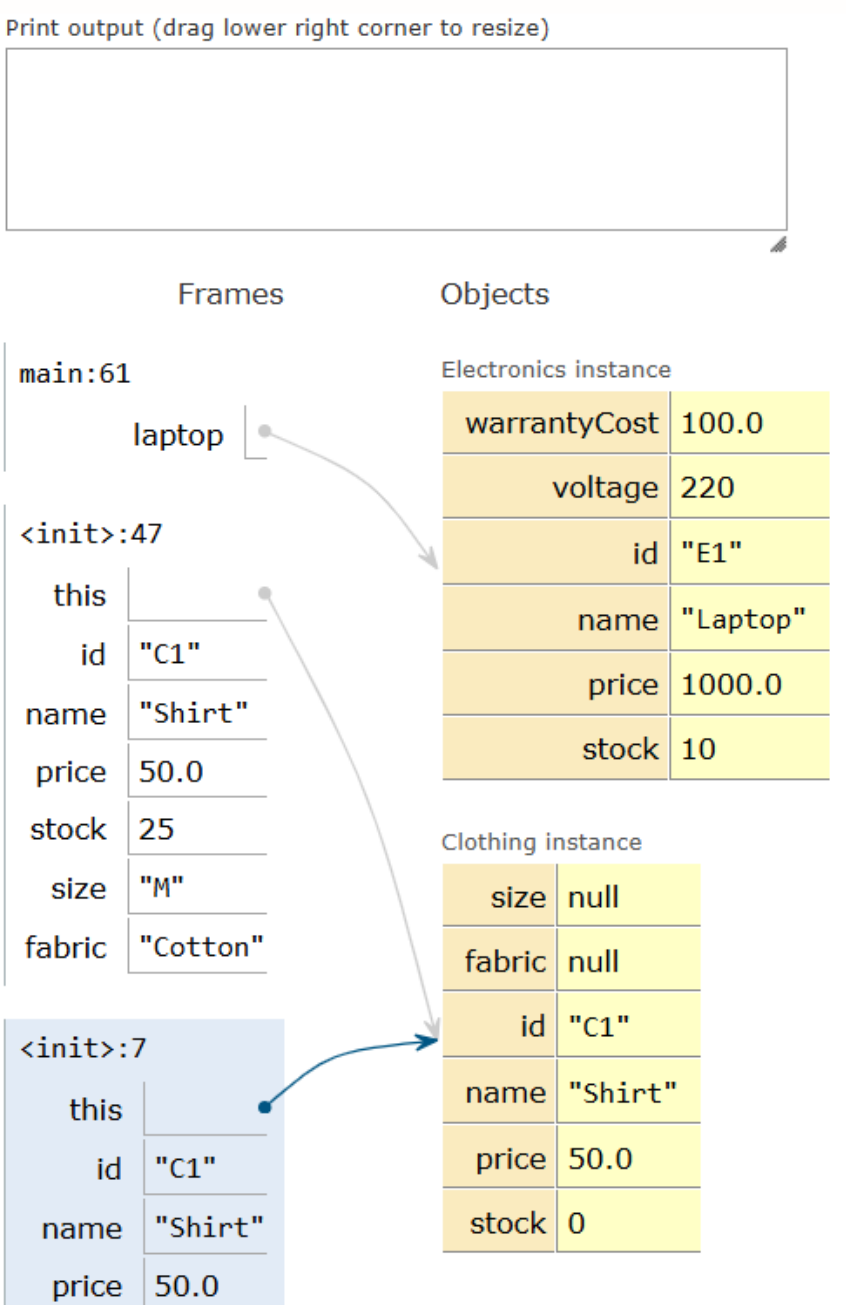

1. The parent constructor assigns the name field of the Clothing instance.
2. The name attribute is assigned a reference to the string 'Shirt'.

### Step 17 of 42: Initializing Clothing Base Price



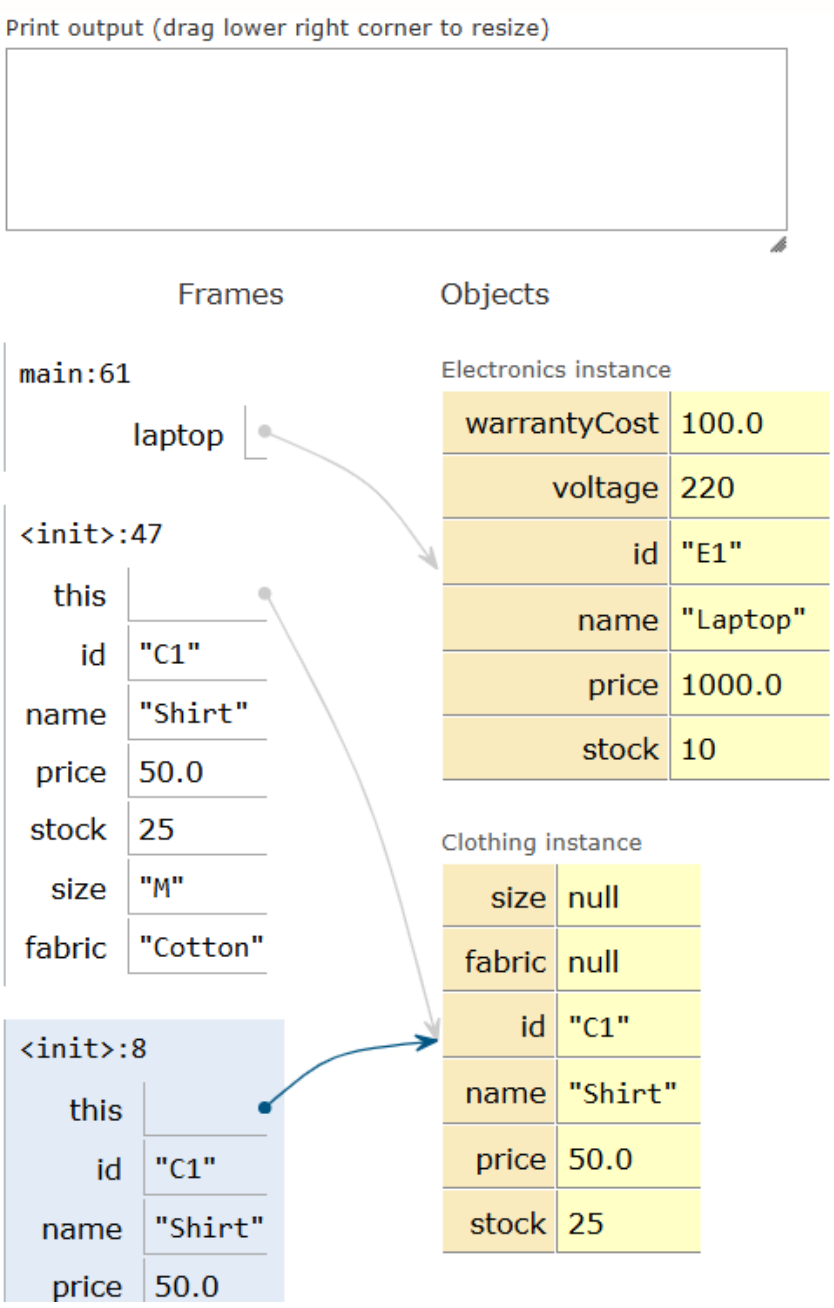

1. The parent constructor assigns this.price = price for the apparel item.
2. The price field on the Clothing heap instance is set to 50.0.

### Step 18 of 42: Initializing Clothing Inventory Stock



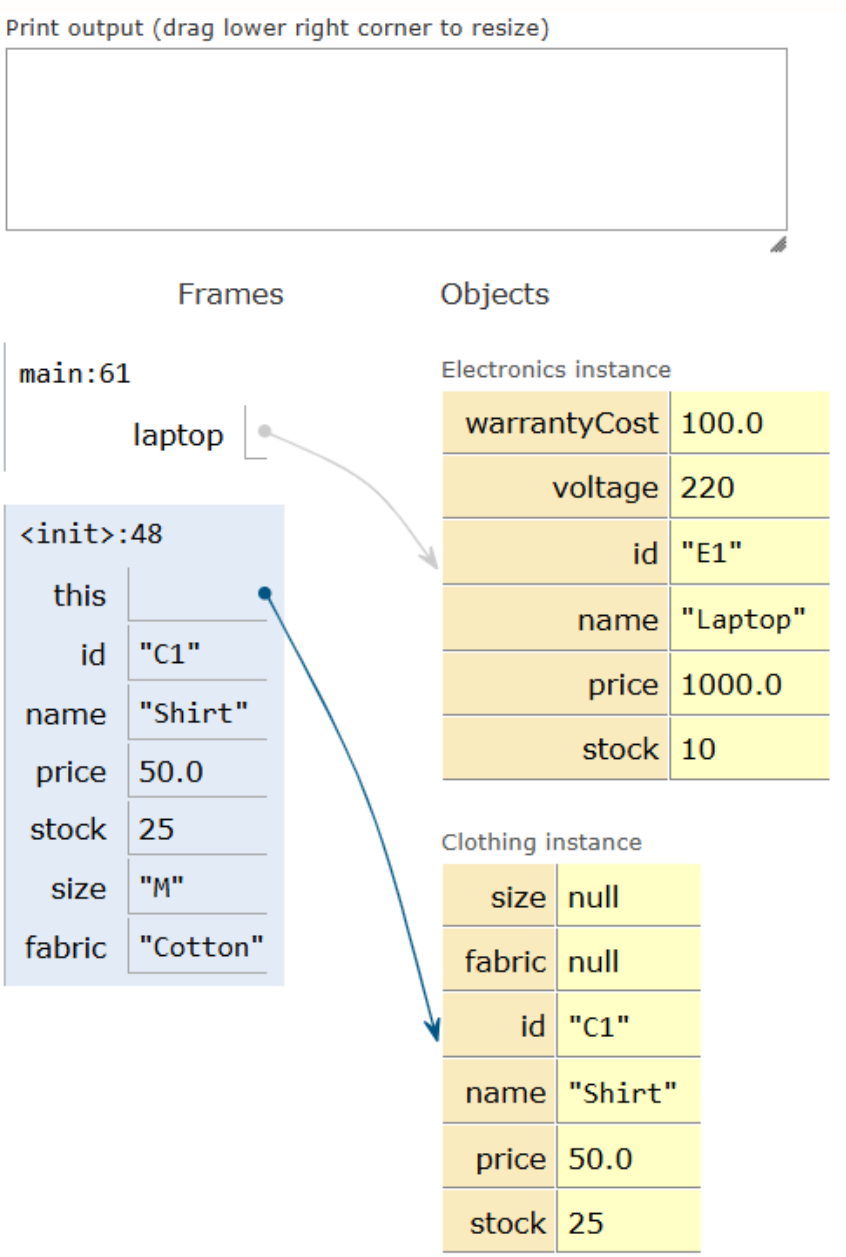

1. The parent constructor sets this.stock = stock for the apparel item.
2. The stock field on the Clothing heap instance is populated with value 25.

### Step 19 of 42: Product Constructor Completion for Clothing



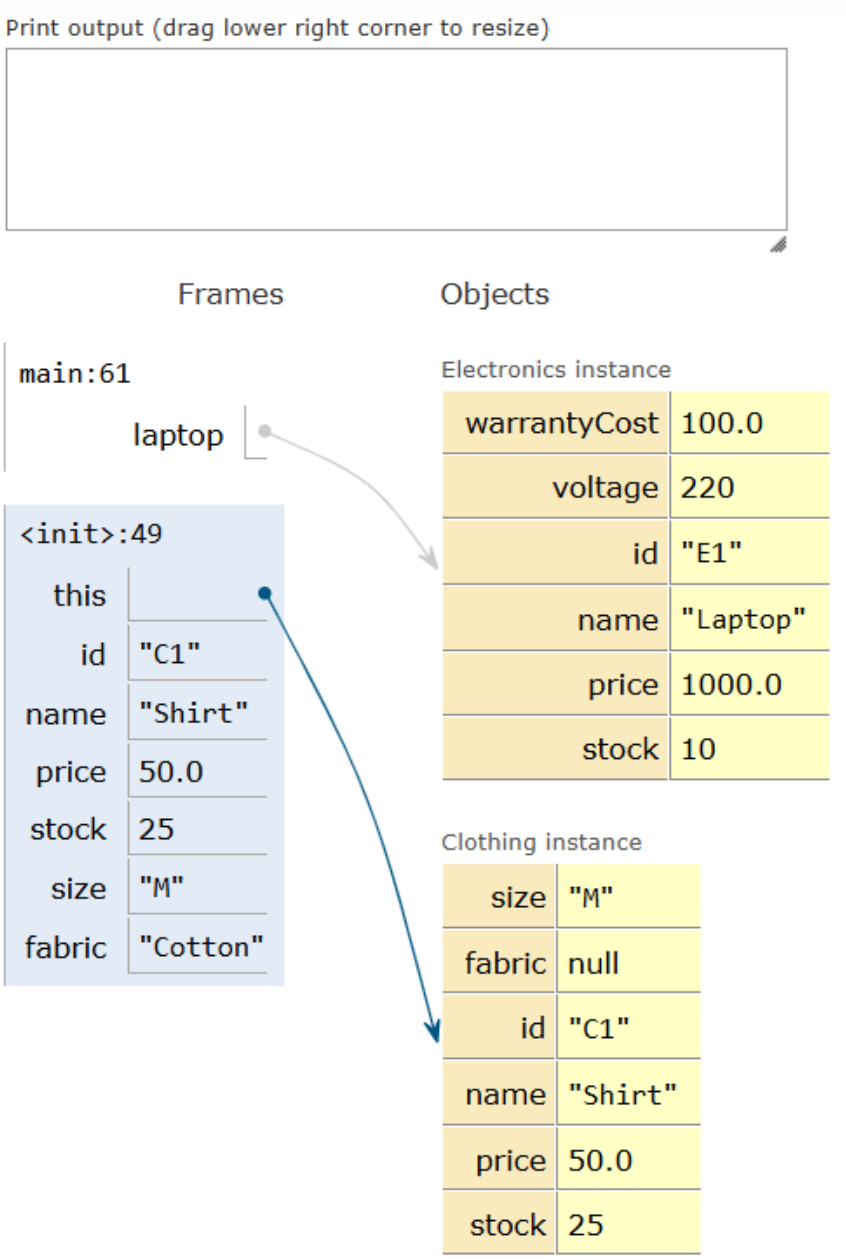

1. The parent Product constructor finishes execution and pops from the call stack.
2. Control returns to the Clothing constructor <init>:47 to initialize apparel-specific fields.

### Step 20 of 42: Initializing Clothing Size Field



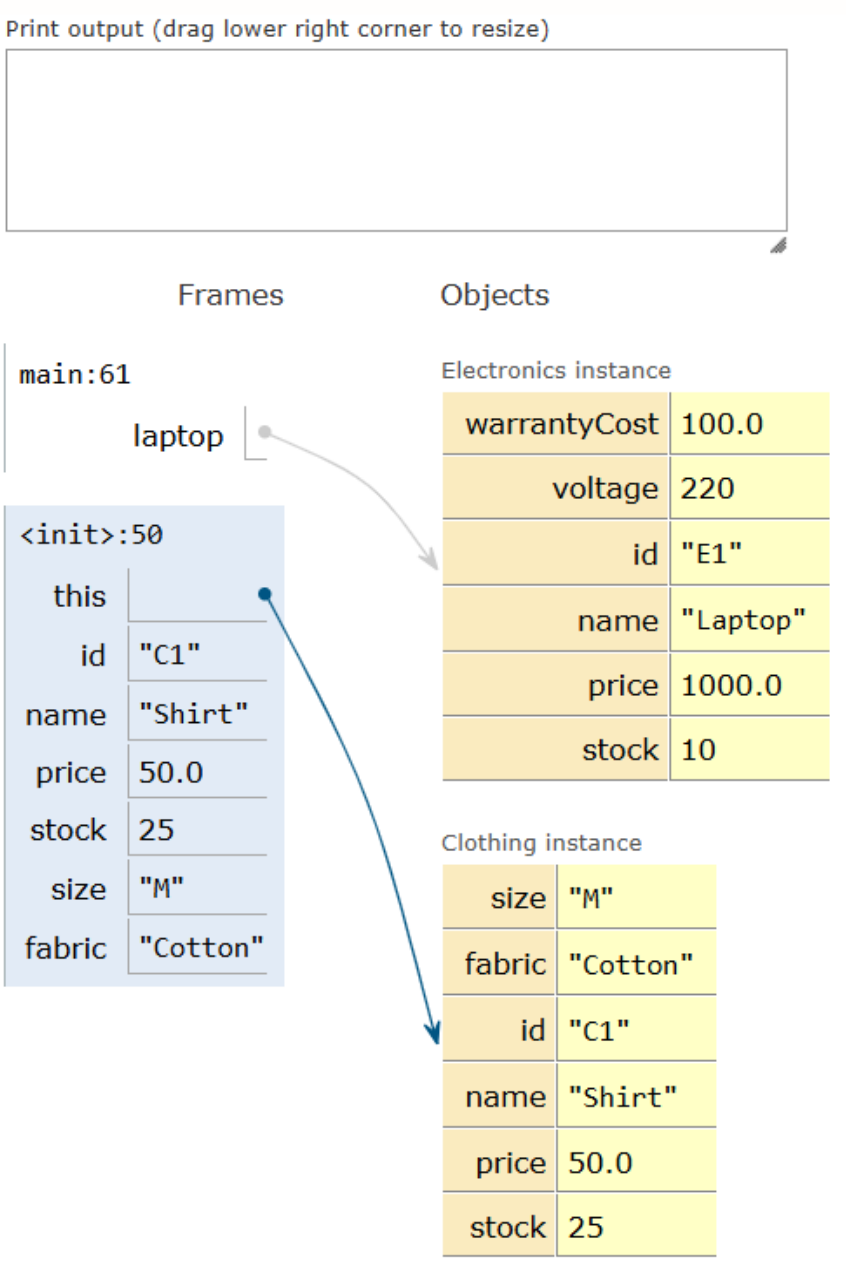

1. The Clothing constructor assigns this.size = size to record apparel sizing.
2. The size field of the heap instance is updated with reference to string 'M'.

### Step 21 of 42: Initializing Clothing Fabric Field



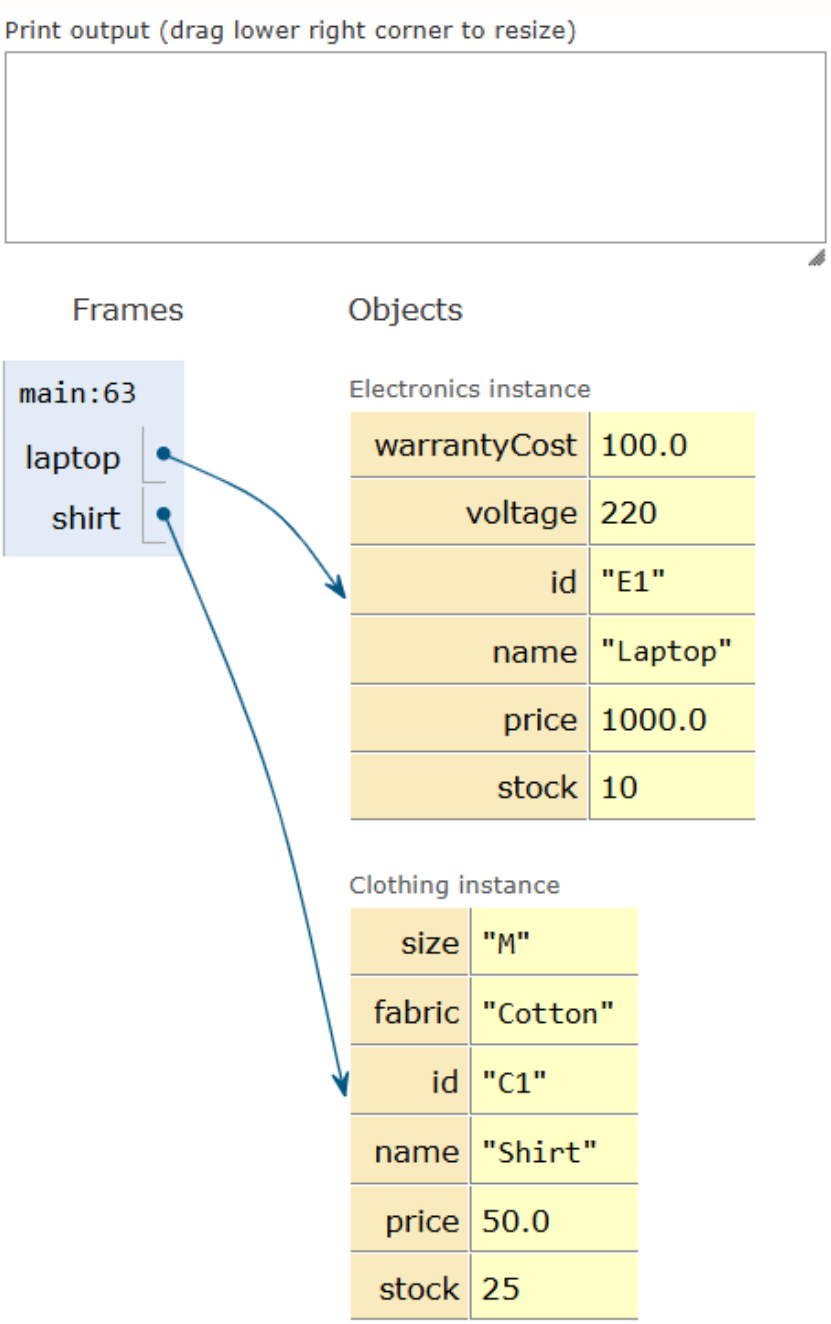

1. The Clothing constructor assigns this.fabric = fabric to record material details.
2. The fabric field of the heap instance is updated with reference to string 'Cotton'.

### Step 22 of 42: Clothing Constructor Completion & Return



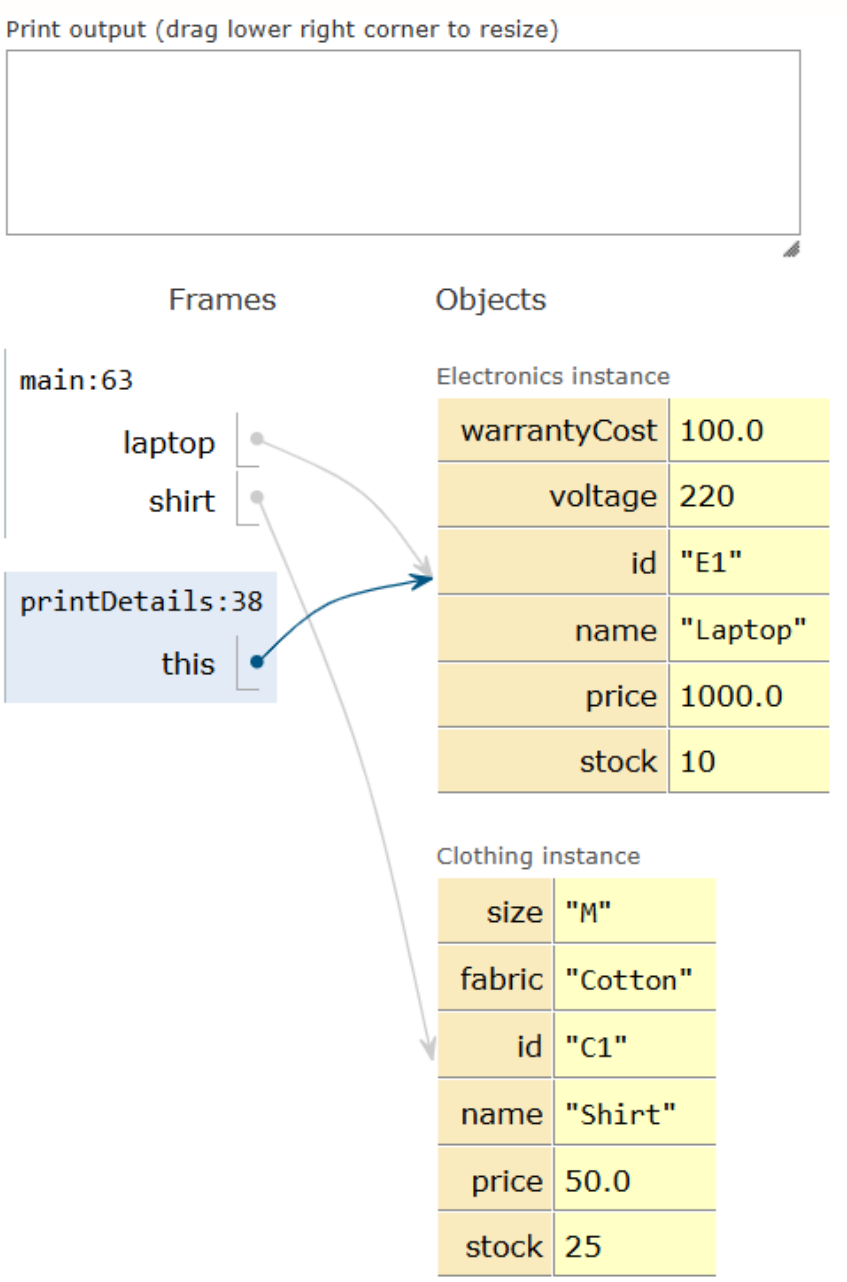

1. The Clothing constructor completes and pops from the stack, returning the object reference.
2. Local variable shirt in the main stack frame now points to the fully initialized clothing instance.

### Step 23 of 42: Invoking laptop.printDetails()



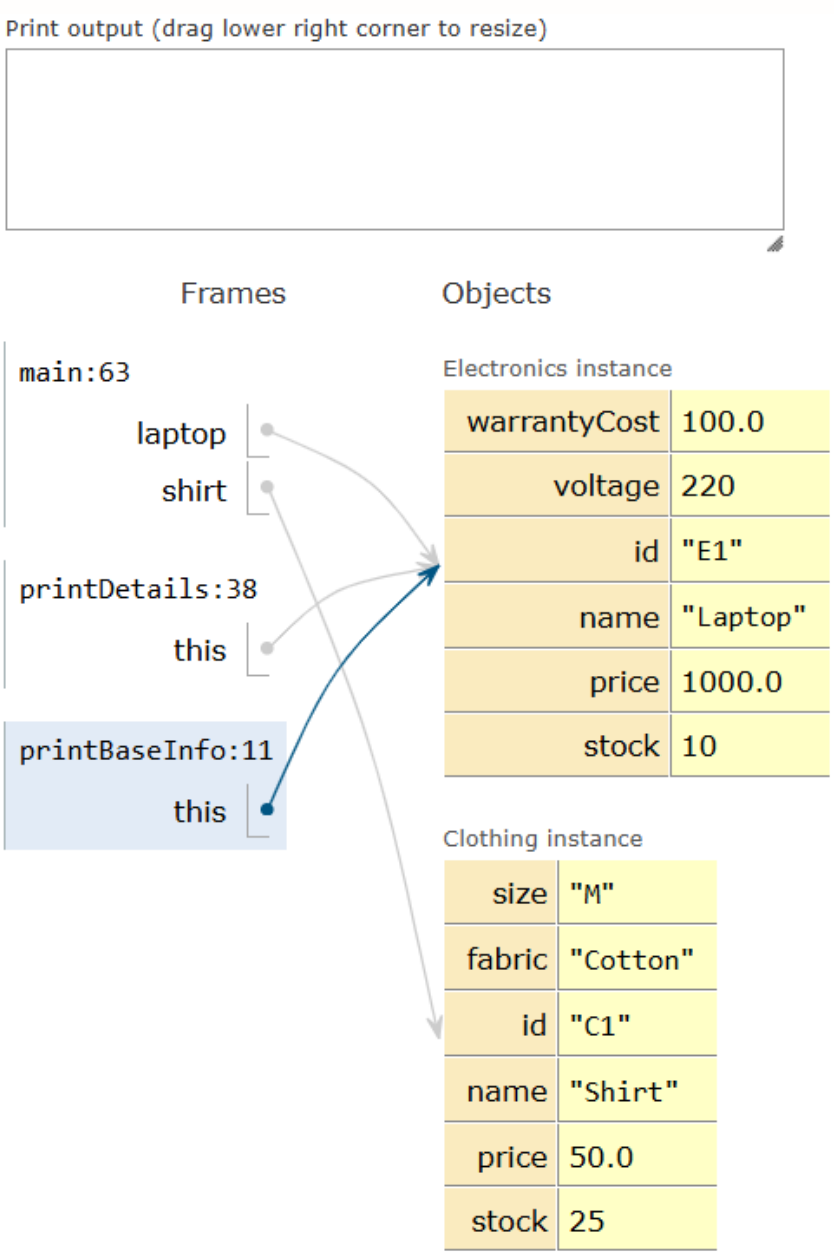

1. Main method execution advances to line 63 to call laptop.printDetails().
2. Dynamic method dispatch begins on the Electronics instance referenced by laptop.

**Step 24 of 42: Electronics printDetails Frame & Parent Method Call**



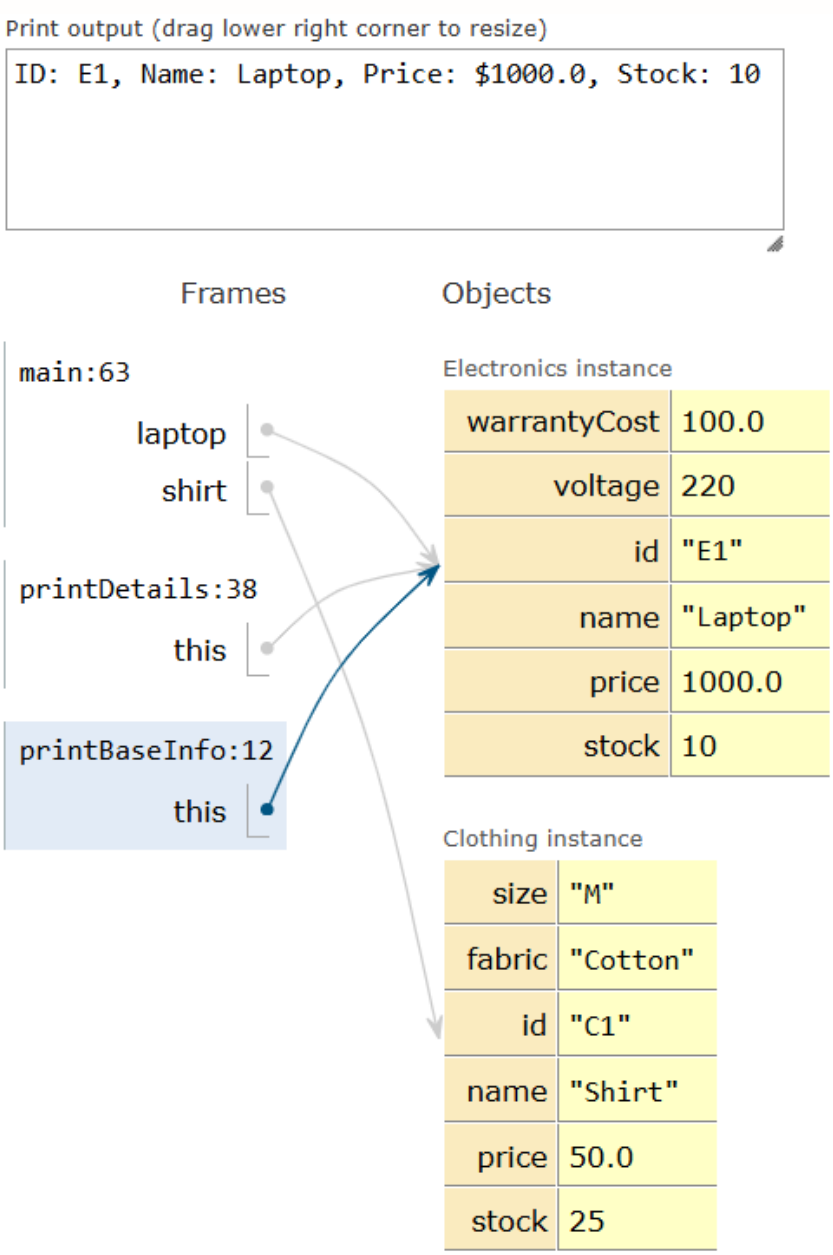

1. The printDetails():38 method frame is allocated on the call stack.
2. Line 38 executes super.printBaseInfo() to invoke the parent class display method.

### Step 25 of 42: Product printBaseInfo Frame via super



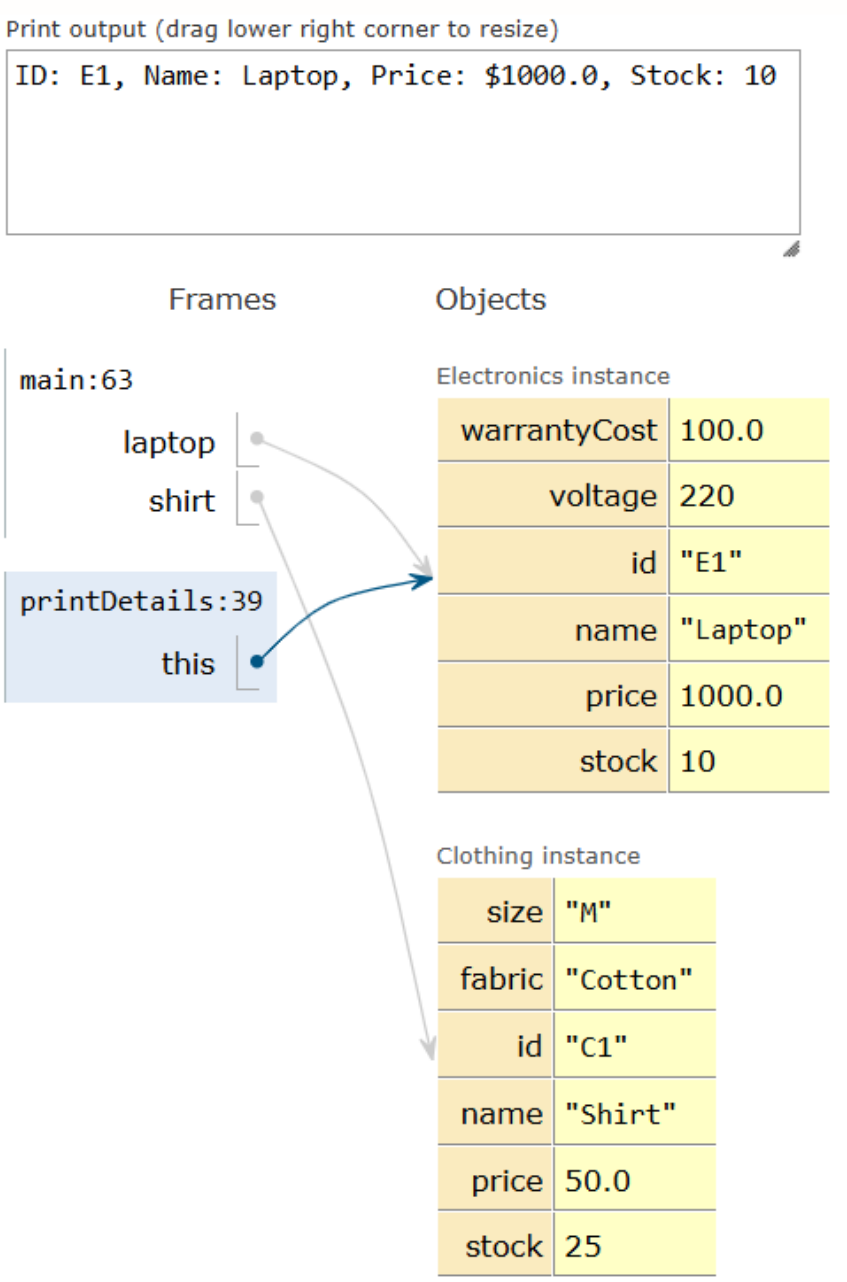

1. The printBaseInfo():11 frame from Product is pushed onto the stack via super.
2. The this pointer in printBaseInfo() correctly references the Electronics instance in the heap.

### Step 26 of 42: Printing Base Info for Electronics



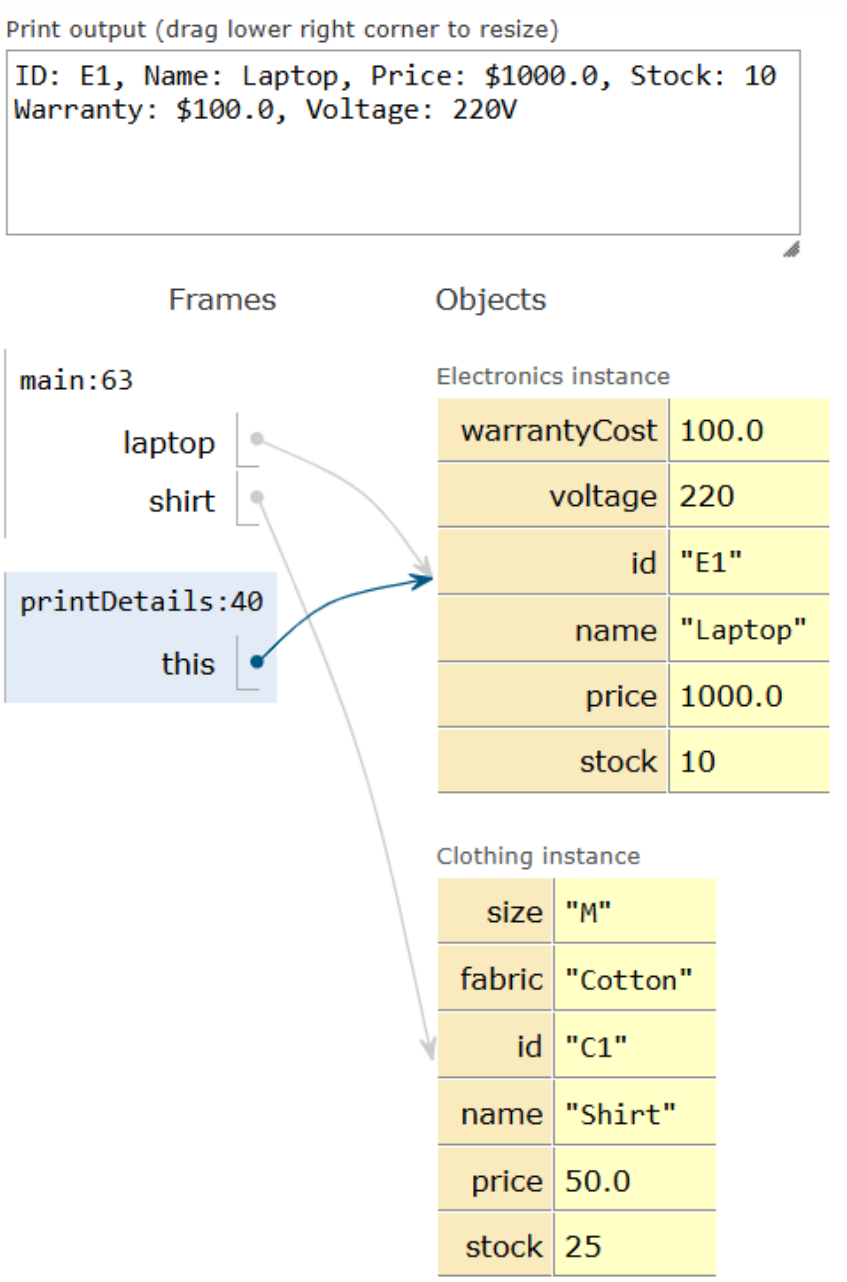

1. System.out.println prints base properties: ID: E1, Name: Laptop, Price: $1000.0, Stock: 10.
2. The output line appears in the standard console output window.

### Step 27 of 42: Return from super.printBaseInfo()




1. Product.printBaseInfo() completes and its stack frame is popped.
2. Execution resumes inside Electronics.printDetails() at line 39 to print subclass properties.

### Step 28 of 42: Printing Subclass Electronics Specifics

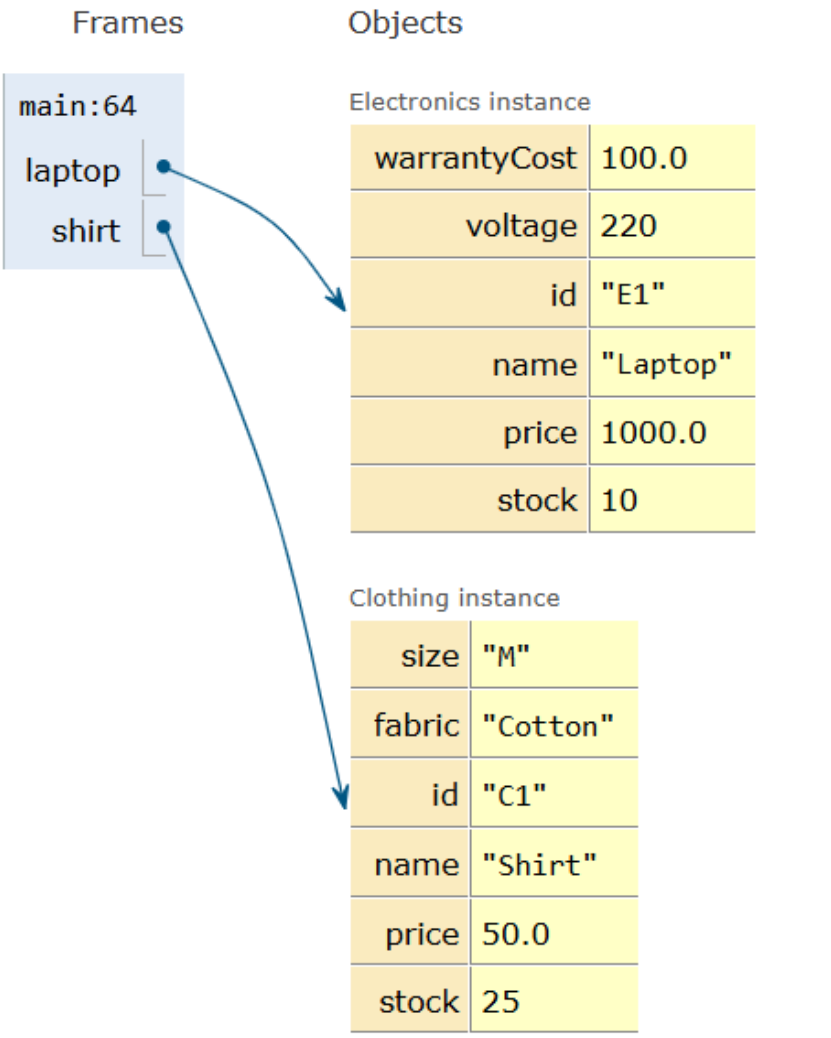



1. Line 39 executes System.out.println to output warranty cost and voltage.
2. String 'Warranty: $100.0, Voltage: 220V' is appended to the console output.

### Step 29 of 42: Completion of Electronics printDetails()

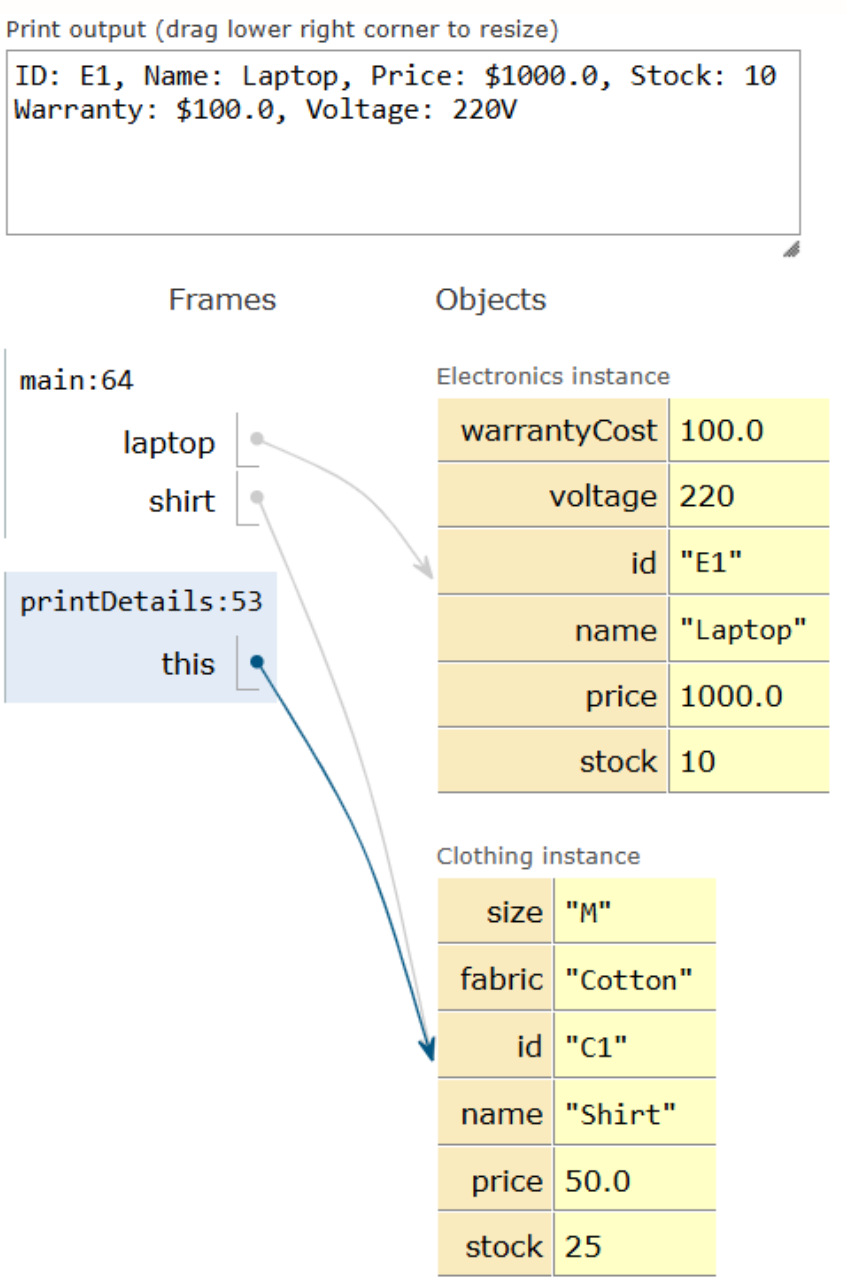
Print output (drag lower right corner to resize)

```
ID: E1, Name: Laptop, Price: $1000.0, Stock: 10
Warranty: $100.0, Voltage: 220V
```



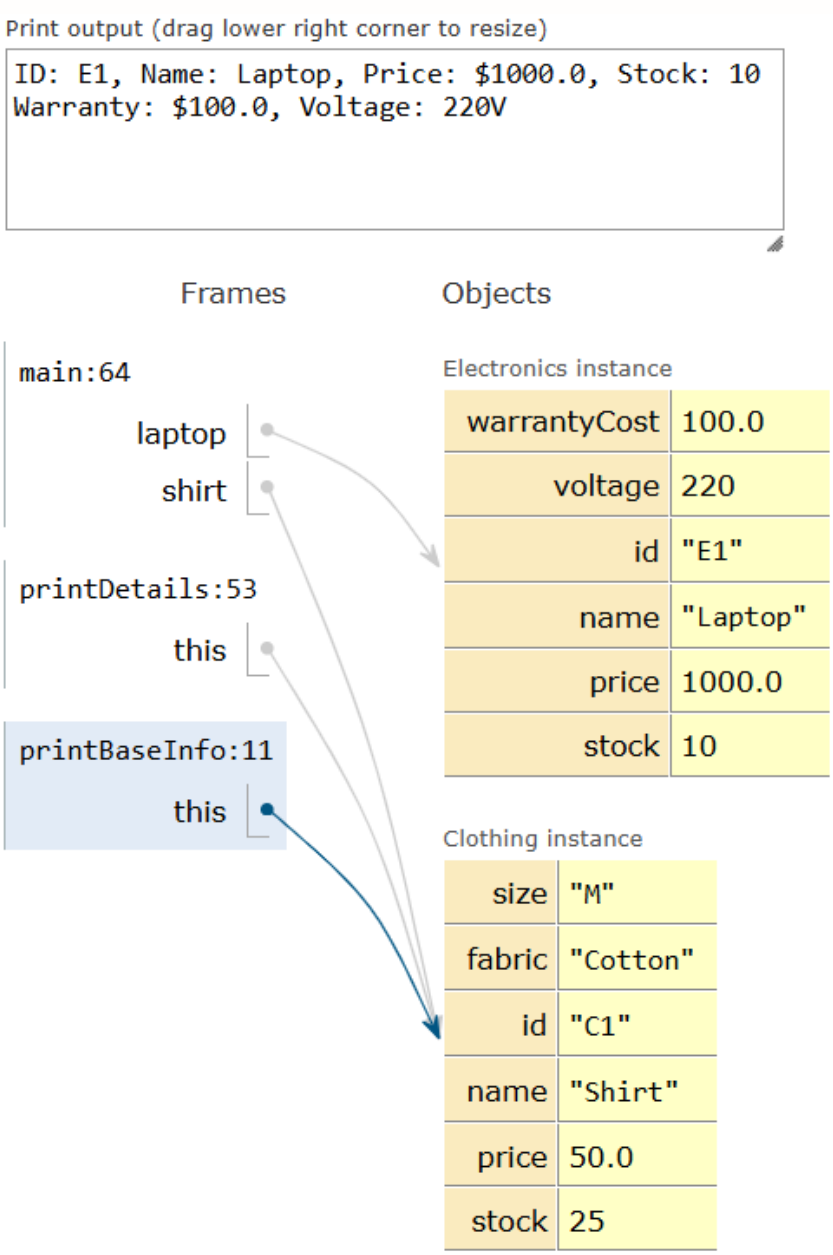

1. All print statements for the laptop are complete in the console.
2. The Electronics.printDetails() method frame prepares to return and pop from the stack.

**Step 30 of 42: Return to Main & Invoking shirt.printDetails()**



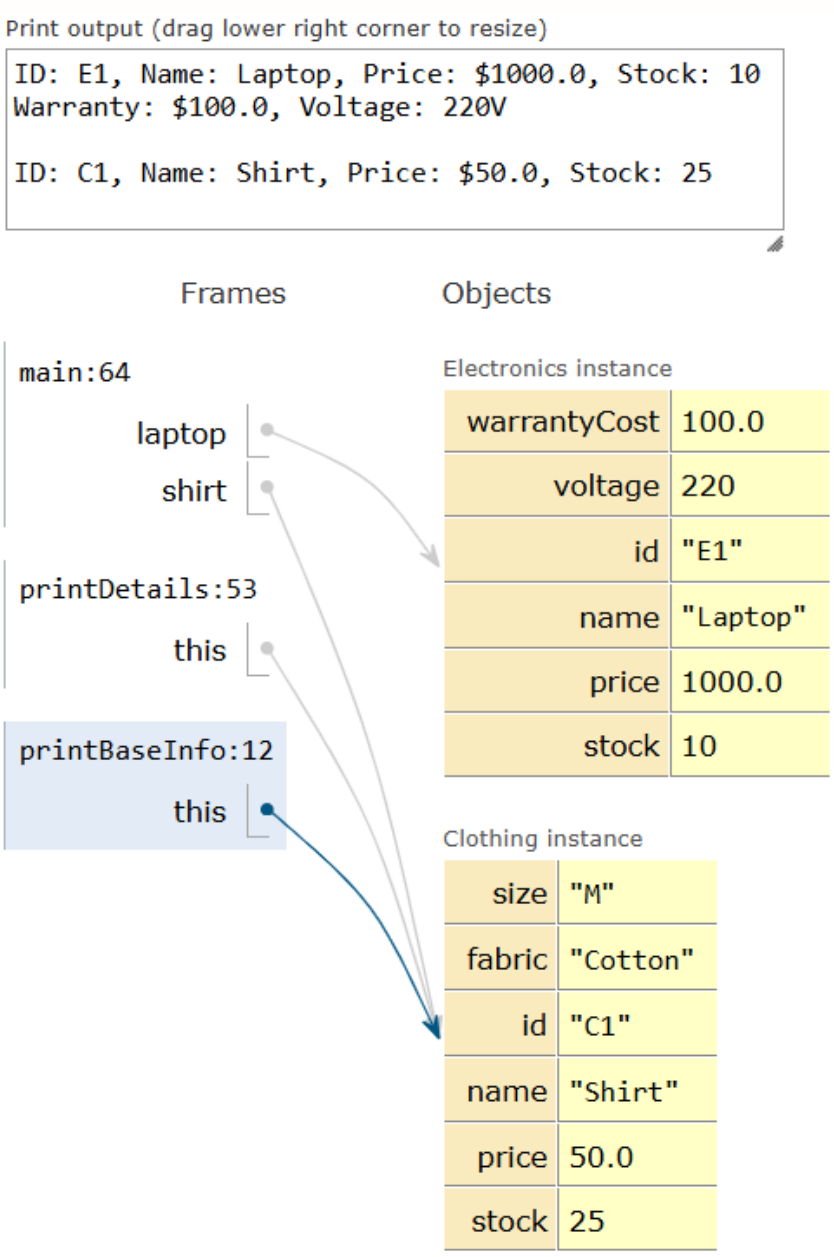

1. Control returns to main() at line 64.
2. The program prepares to call shirt.printDetails() on the Clothing instance.

### Step 31 of 42: Clothing printDetails Frame & Parent Method Call




1. The Clothing.printDetails():53 method frame is pushed onto the call stack.
2. Line 53 invokes super.printBaseInfo() to reuse parent display logic.

### Step 32 of 42: Product printBaseInfo Frame for Clothing



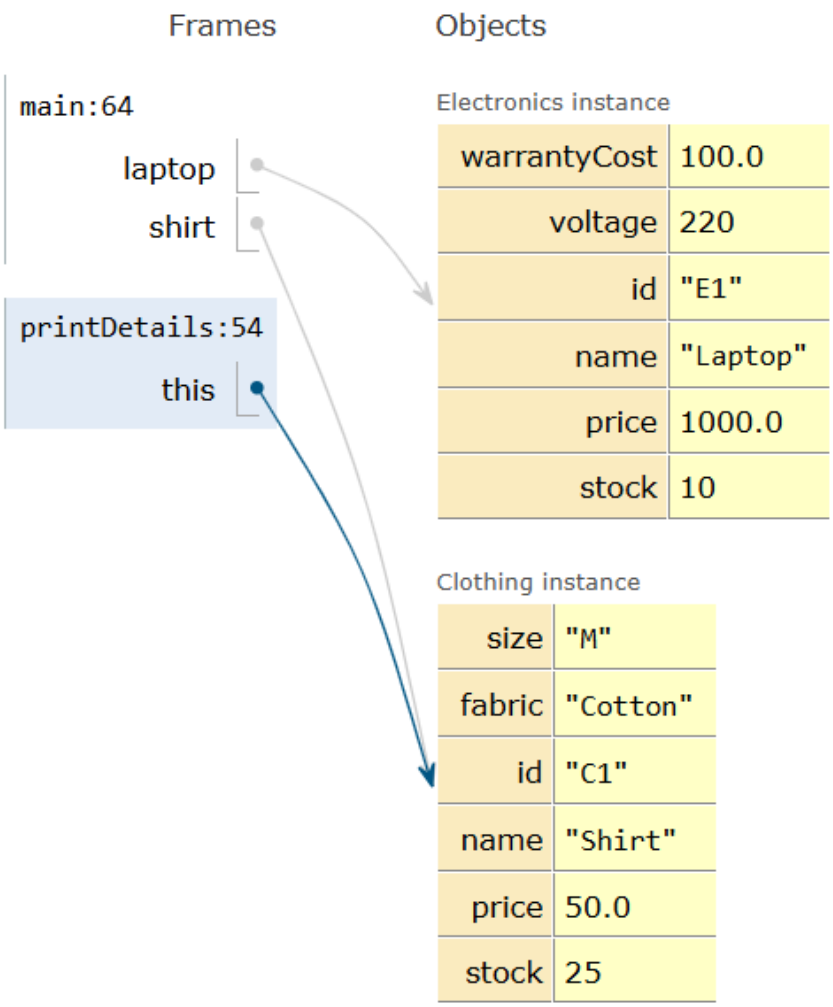

1. The Product.printBaseInfo():11 frame is pushed onto the stack via super.
2. The method accesses the shirt object properties stored on the heap.

### Step 33 of 42: Printing Base Info for Clothing

Print output (drag lower right corner to resize)

```
ID: E1, Name: Laptop, Price: $1000.0, Stock: 10
Warranty: $100.0, Voltage: 220V

ID: C1, Name: Shirt, Price: $50.0, Stock: 25
```



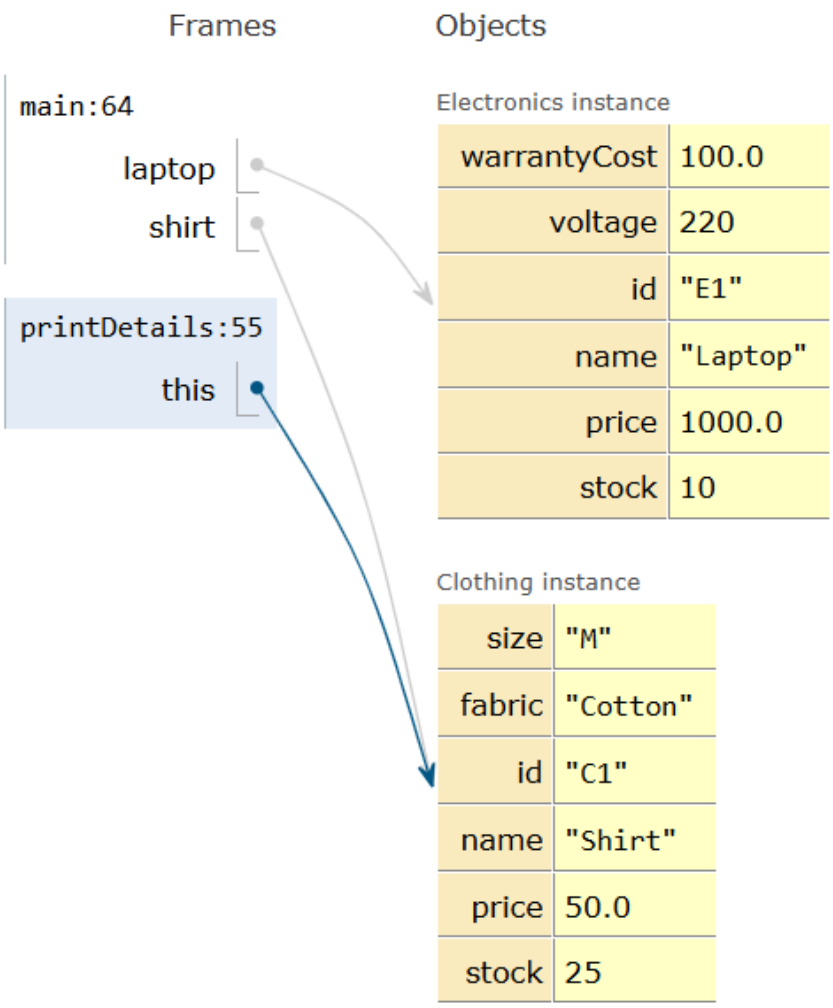

1. System.out.println prints base properties: ID: C1, Name: Shirt, Price: $50.0, Stock: 25.
2. The output is successfully appended to the console output panel.

### Step 34 of 42: Return from Clothing Base Display

Print output (drag lower right corner to resize)

```
ID: E1, Name: Laptop, Price: $1000.0, Stock: 10
Warranty: $100.0, Voltage: 220V

ID: C1, Name: Shirt, Price: $50.0, Stock: 25
Size: M, Fabric: Cotton
```



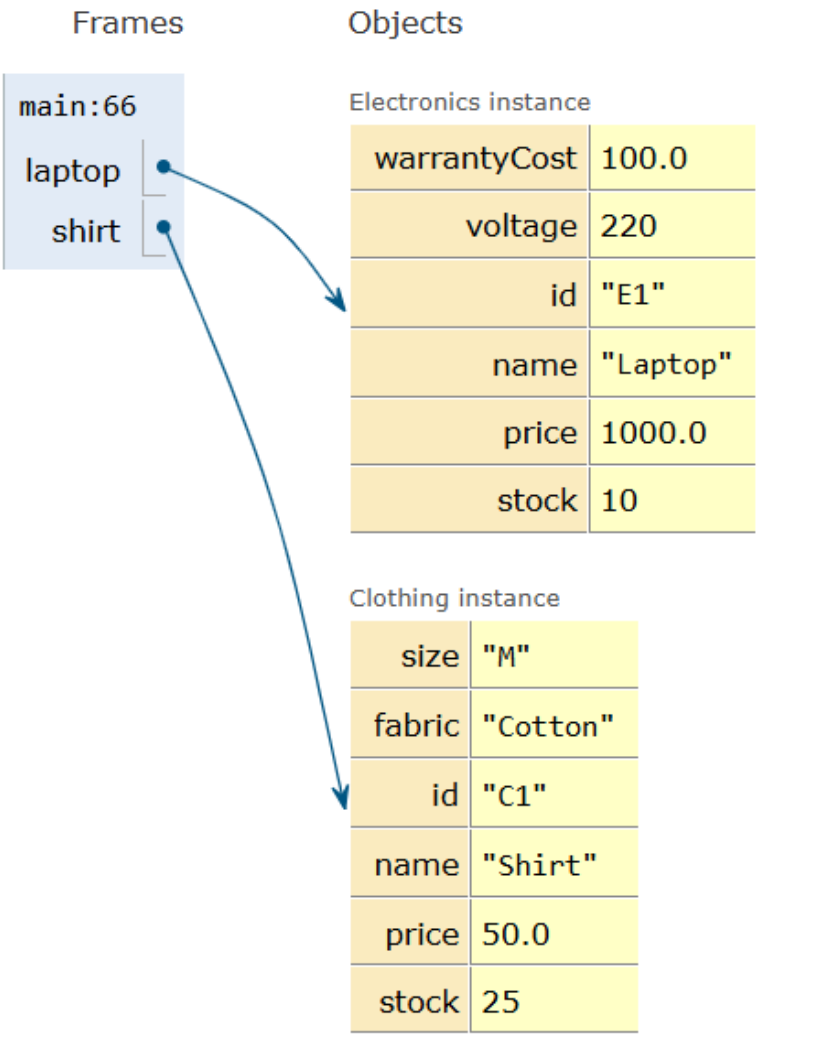

1. printBaseInfo() completes and its stack frame is removed.
2. Control returns to Clothing.printDetails() at line 54 to print clothing-specific attributes.

### Step 35 of 42: Printing Subclass Clothing Specifics

Print output (drag lower right corner to resize)

```
ID: E1, Name: Laptop, Price: $1000.0, Stock: 10
Warranty: $100.0, Voltage: 220V

ID: C1, Name: Shirt, Price: $50.0, Stock: 25
Size: M, Fabric: Cotton
```



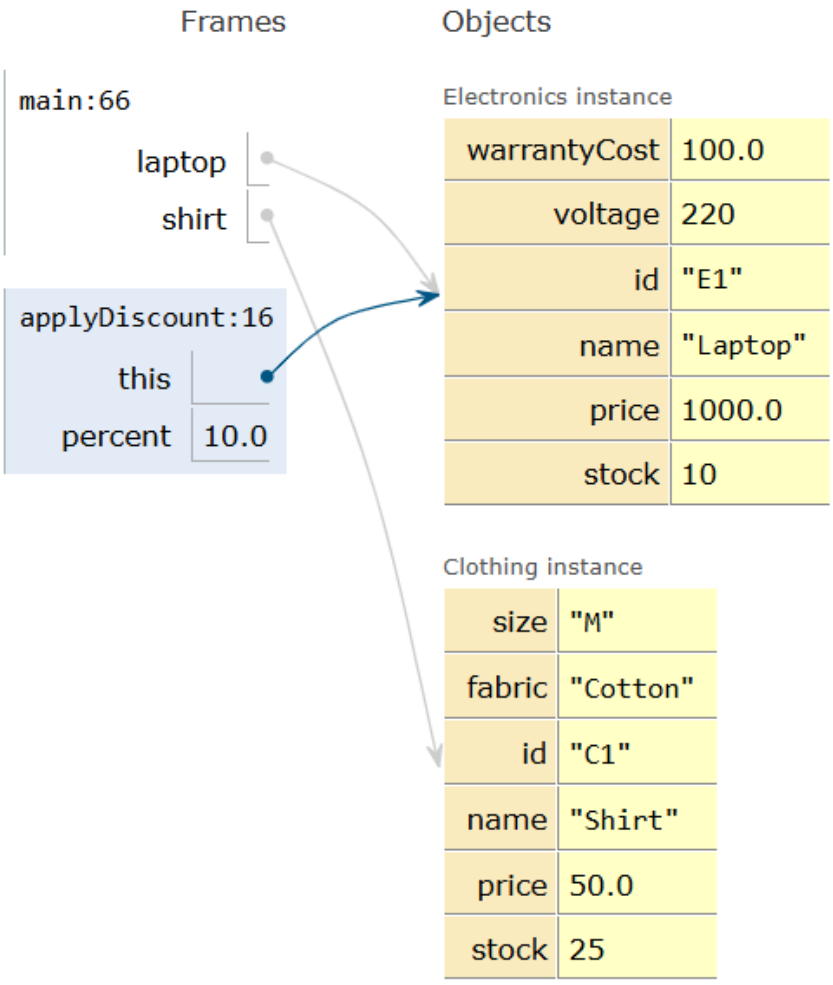

1. Line 54 executes System.out.println to output size and fabric details.
2. String 'Size: M, Fabric: Cotton' is written to standard output.

**Step 36 of 42: Completion of Clothing printDetails()**

Print output (drag lower right corner to resize)

```
ID: E1, Name: Laptop, Price: $1000.0, Stock: 10
Warranty: $100.0, Voltage: 220V

ID: C1, Name: Shirt, Price: $50.0, Stock: 25
Size: M, Fabric: Cotton
```



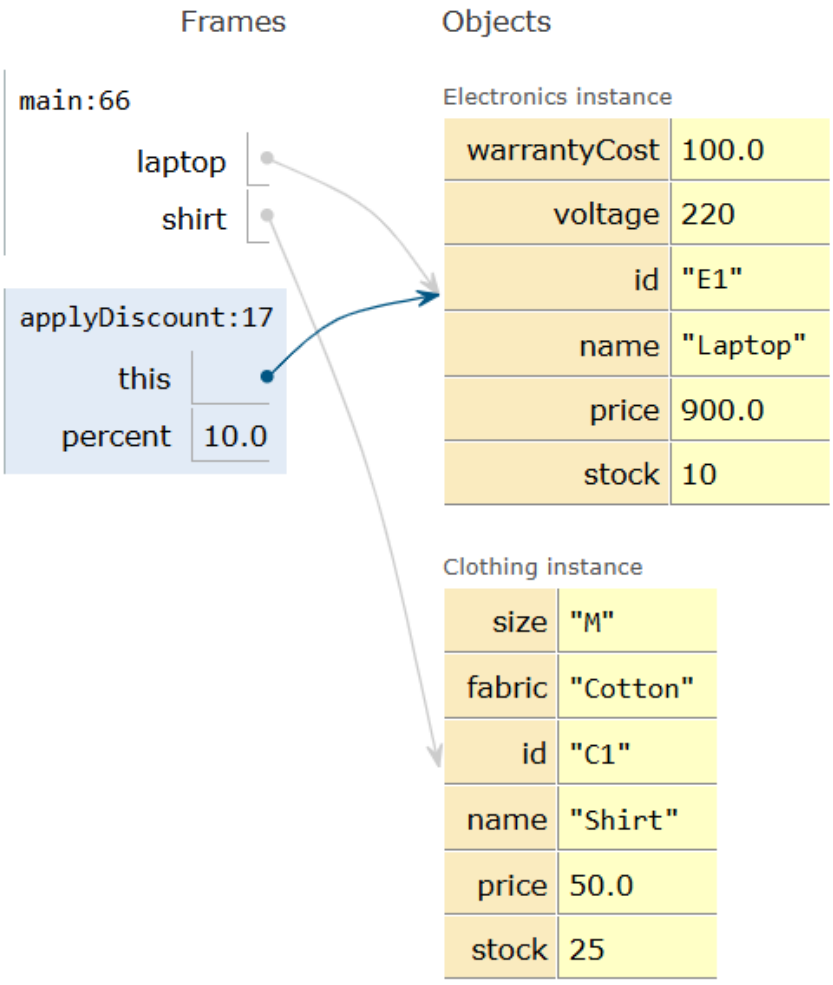

1. All print operations for the clothing item have finished displaying.
2. The Clothing.printDetails() frame prepares to exit and pop from the stack.

### Step 37 of 42: Return to Main & Invoking Percentage Discount

Print output (drag lower right corner to resize)

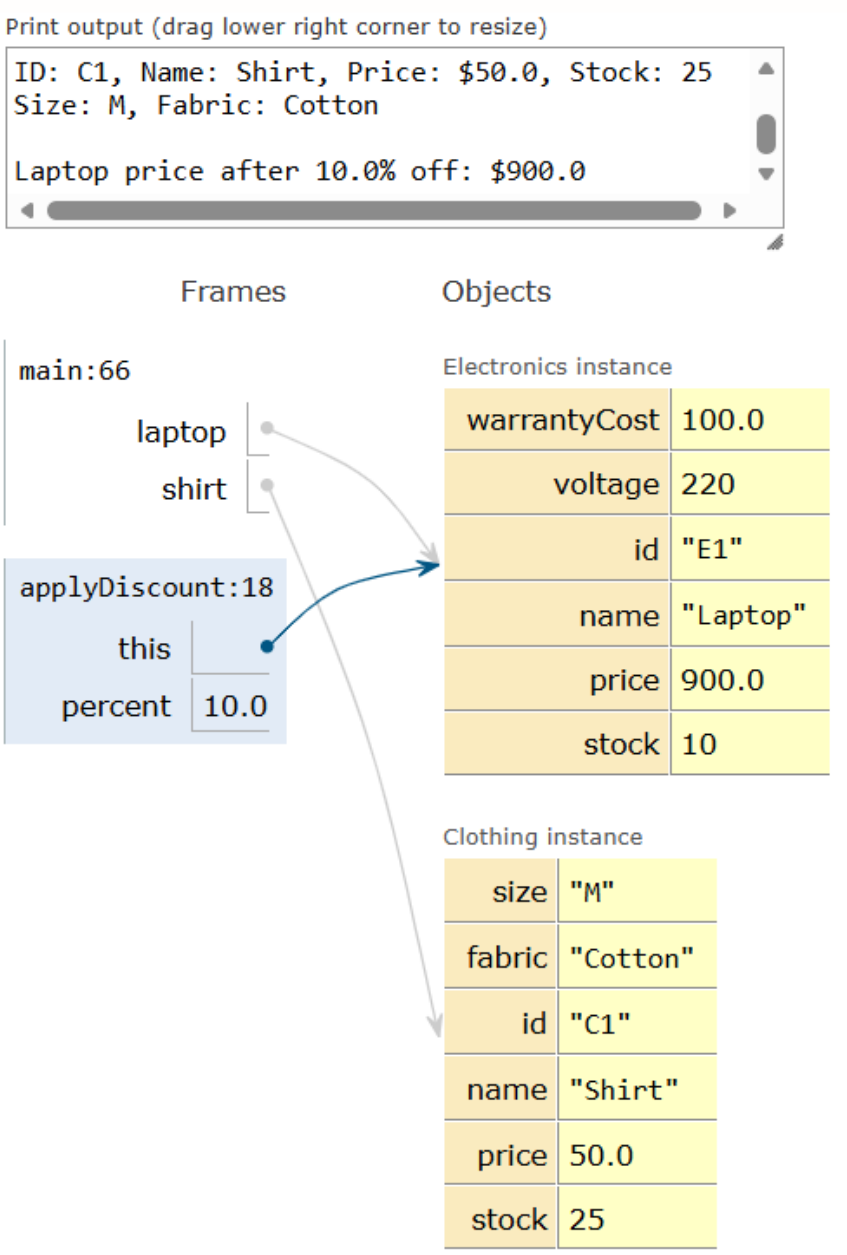
```
ID: E1, Name: Laptop, Price: $1000.0, Stock: 10
Warranty: $100.0, Voltage: 220V

ID: C1, Name: Shirt, Price: $50.0, Stock: 25
Size: M, Fabric: Cotton
```



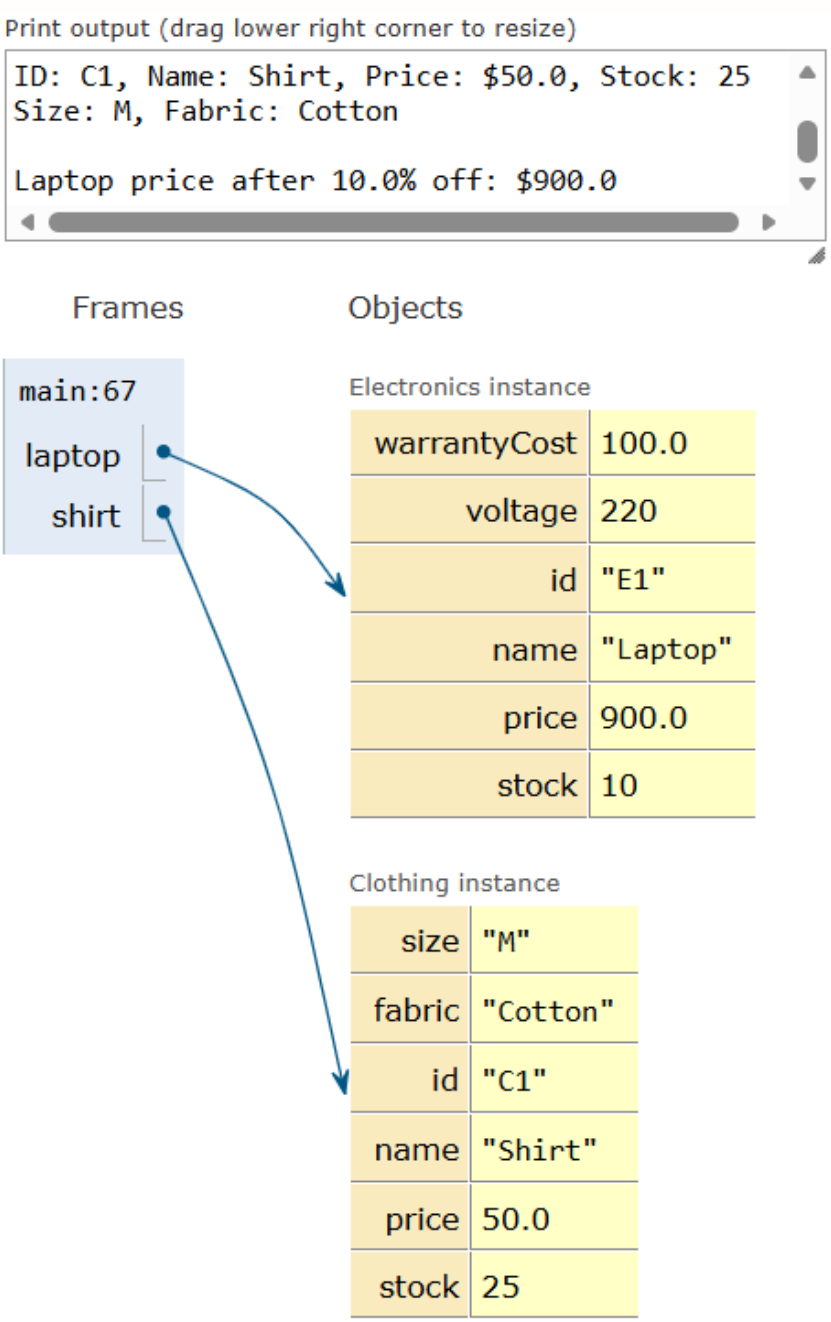

1. Control returns to main() at line 66.
2. Execution prepares to call overloaded discount method: laptop.applyDiscount(10.0).

### Step 38 of 42: Executing Overload 1 (Percentage Discount)



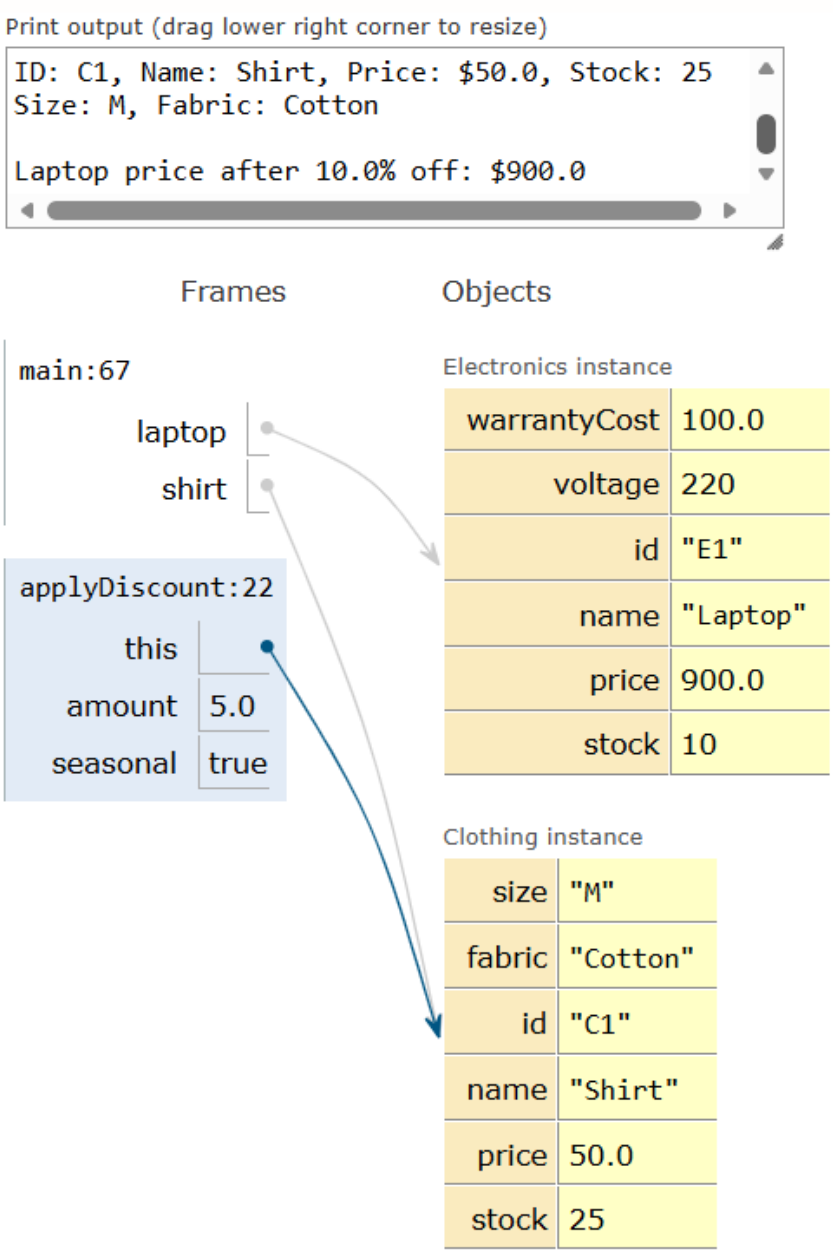

1. The frame for applyDiscount(double percent) is pushed onto the stack with percent=10.0.
2. The discount calculation evaluates: price -= price * (10.0 / 100.0).

### Step 39 of 42: Updating Laptop Price & Outputting Discount



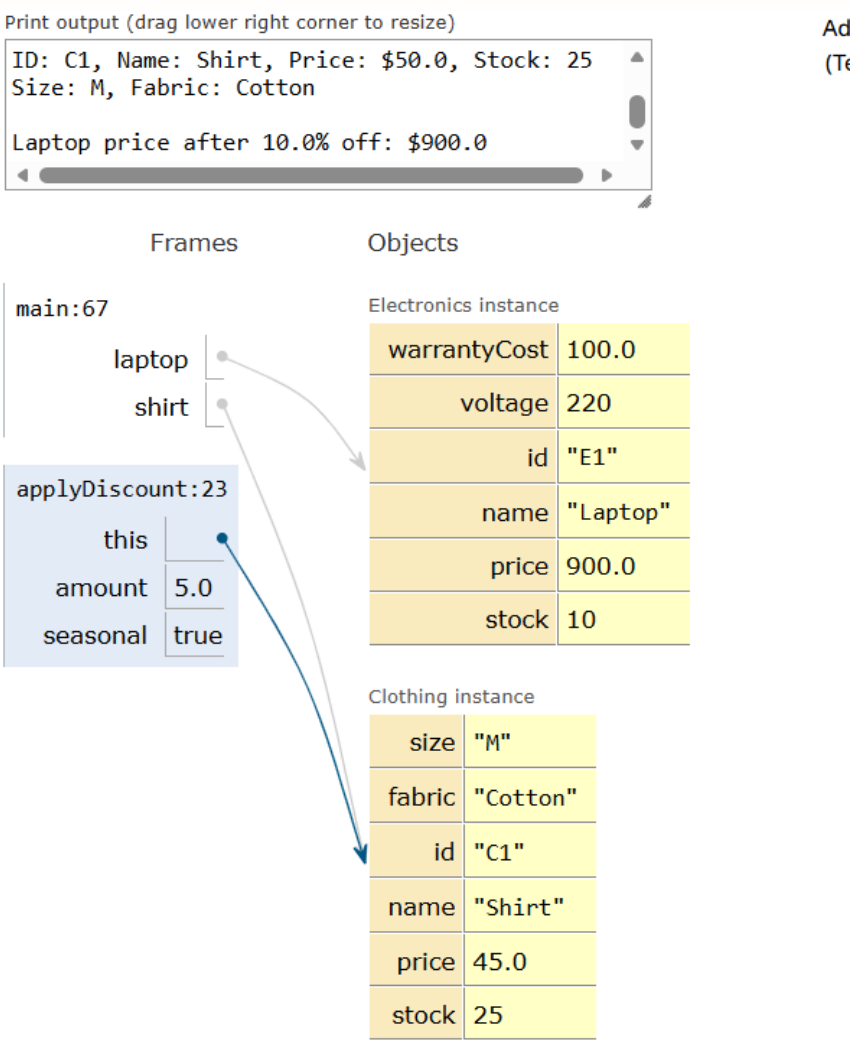

1. The price field of the Electronics heap object updates from $1000.0 to $900.0.
2. Console outputs: 'Laptop price after 10.0% off: $900.0'.

**Step 40 of 42: Return to Main & Invoking Seasonal Discount**



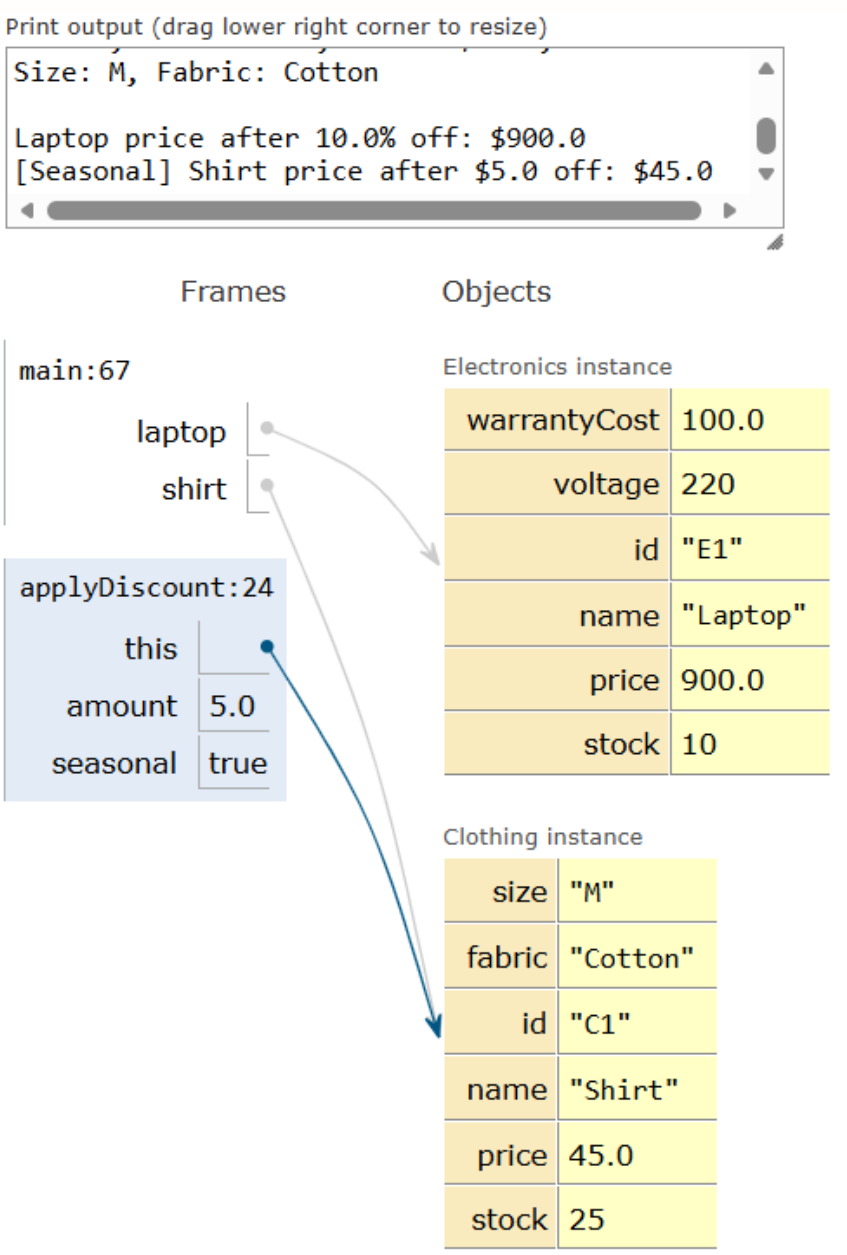

1. Overload 1 completes, pops from the stack, and returns to main() at line 67.
2. Execution prepares to call overloaded discount method: shirt.applyDiscount(5.0, true).

### Step 41 of 42: Executing Overload 2 (Flat Seasonal Discount)




1. The frame for applyDiscount(double amount, boolean seasonal) is pushed onto the stack.
2. The flat amount $5.0 is subtracted from the price, updating shirt price on heap to $45.0.

### Step 42 of 42: Outputting Seasonal Discount & Program Termination




1. Console outputs seasonal reduction: '[Seasonal] Shirt price after $5.0 off: $45.0'.
2. All method frames terminate cleanly, completing the program execution.

## 4. Complete Java Source Code

```
class Product {
    private String id, name;
    private double price;
    private int stock;

    public Product(String id, String name, double price, int stock) {
        this.id = id; this.name = name; this.price = price; this.stock = stock;
    }

    public void printBaseInfo() {
        System.out.println("ID: " + id + ", Name: " + name + ", Price: $" + price + ",
Stock: " + stock);
    }

    // Overload 1: Percentage discount
    public void applyDiscount(double percent) {
        price -= price * (percent / 100.0);
        System.out.println(name + " price after " + percent + "% off: $" + price);
    }

    // Overload 2: Flat amount discount with seasonal flag
    public void applyDiscount(double amount, boolean seasonal) {
        price -= amount;
        System.out.println((seasonal ? "[Seasonal] " : "") + name + " price after $" +
amount + " off: $" + price);
    }
}

class Electronics extends Product {
    private double warrantyCost;
    private int voltage;

    public Electronics(String id, String name, double price, int stock, double
warrantyCost, int voltage) {
        super(id, name, price, stock); // Constructor chaining
        this.warrantyCost = warrantyCost;
        this.voltage = voltage;
    }

    public void printDetails() {
        super.printBaseInfo(); // Parent method invocation using super
        System.out.println("Warranty: $" + warrantyCost + ", Voltage: " + voltage +
"V\n");
    }
}

class Clothing extends Product {
    private String size, fabric;

    public Clothing(String id, String name, double price, int stock, String size,
String fabric) {
        super(id, name, price, stock); // Constructor chaining
        this.size = size;
```

```
        this.fabric = fabric;
    }

    public void printDetails() {
        super.printBaseInfo(); // Parent method invocation using super
        System.out.println("Size: " + size + ", Fabric: " + fabric + "\n");
    }
}

public class Main {
    public static void main(String[] args) {
        Electronics laptop = new Electronics("E1", "Laptop", 1000.0, 10, 100.0, 220);
        Clothing shirt = new Clothing("C1", "Shirt", 50.0, 25, "M", "Cotton");

        laptop.printDetails();
        shirt.printDetails();

        laptop.applyDiscount(10.0);
        shirt.applyDiscount(5.0, true);
    }
}
```

## 5. Program Execution Console Output

```
ID: E1, Name: Laptop, Price: $1000.0, Stock: 10
Warranty: $100.0, Voltage: 220V

ID: C1, Name: Shirt, Price: $50.0, Stock: 25
Size: M, Fabric: Cotton

Laptop price after 10.0% off: $900.0
[Seasonal] Shirt price after $5.0 off: $45.0
```